Computer Vision 1

HC4a

Global Transforms & Image Stitching

Lecturer: Yen-Chia Hsu

Sep 2026

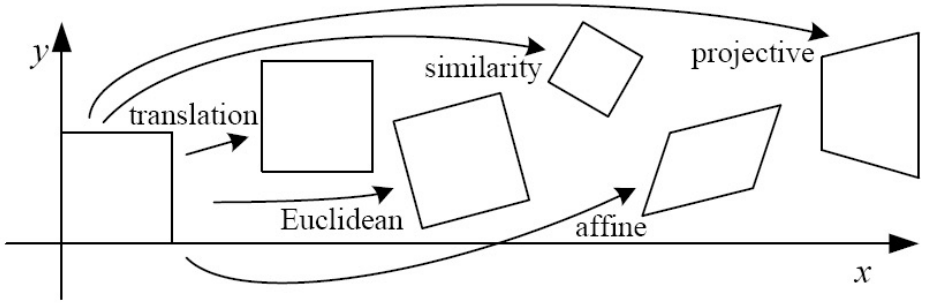
Credit: This slide is inherited from Dr. Martin Oswald and Dr. Dimitris Tzionas



University of Amsterdam

Agenda

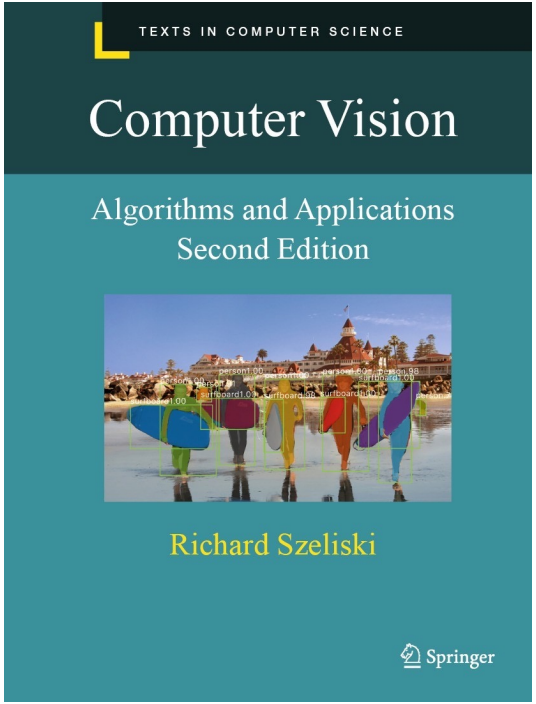
- Global operations
  - Global Transformations
  - Image Warping
- Image Stitching
  - Transformation computation
    - Reprojection model
    - Key point matching
    - RANSAC
    - Solving transformation
  - Blending





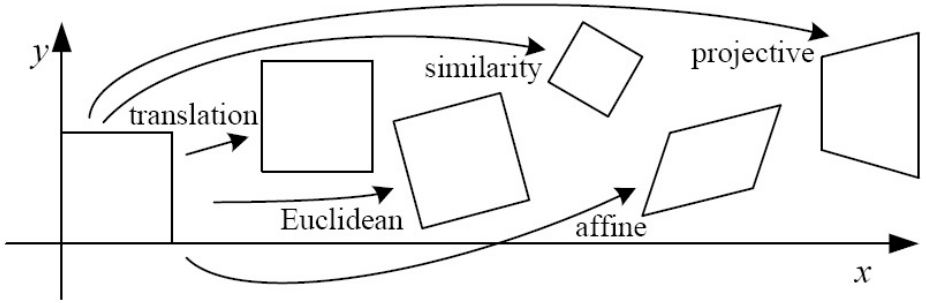
Textbook

- 8.1
- 8.2
- 2.1



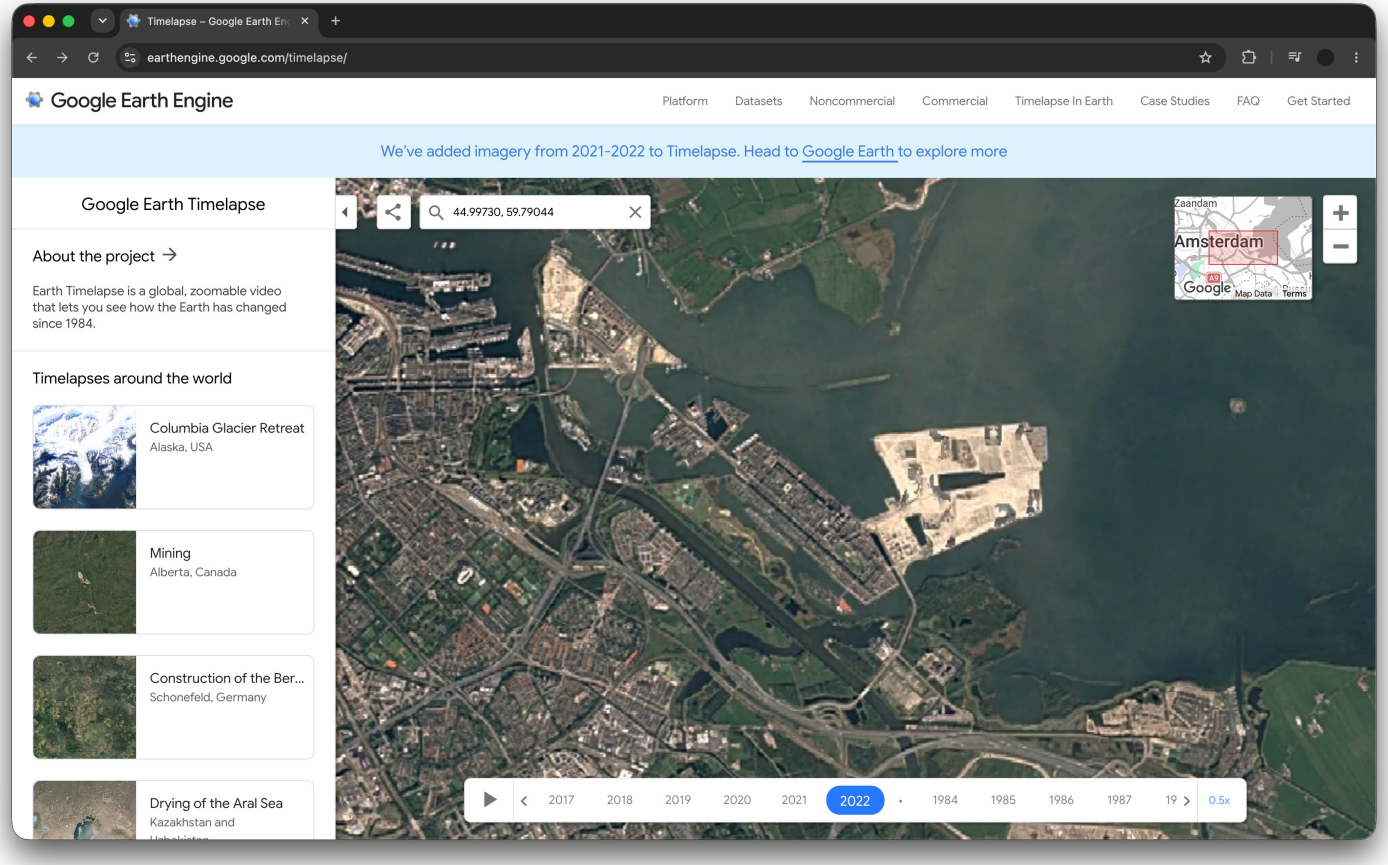
Agenda

- Global operations
  - Global Transformations
  - Image Warping
- Image Stitching
  - Transformation computation
    - Reprojection model
    - Key point matching
    - RANSAC
    - Solving transformation
  - Blending



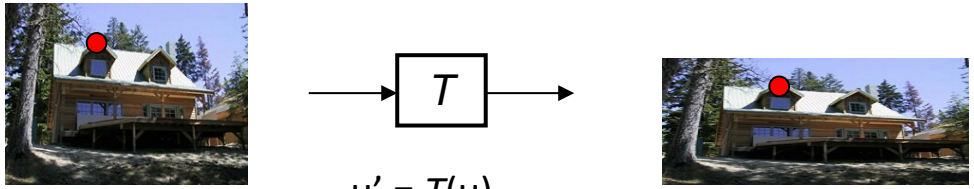


Motivation: stitch satellite images



source -- https://earthengine.google.com/timelapse#v=52.35455,4.98145,11.716,latLng&t=3.83&ps=50&bt=19840101&et=20221231

Parametric (Global) Transformations



What does it mean that *T* is global?

- *T* is the same for any point p in the entire image domain

Usually, *T* can be described by just a few numbers (parameters)

For linear transformations, we can represent T as a matrix

$$u' = \mathbf{T}u$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix}$$

Common Linear Transformations



Original

Transformed



Translation



Rotation



Scaling



Shear

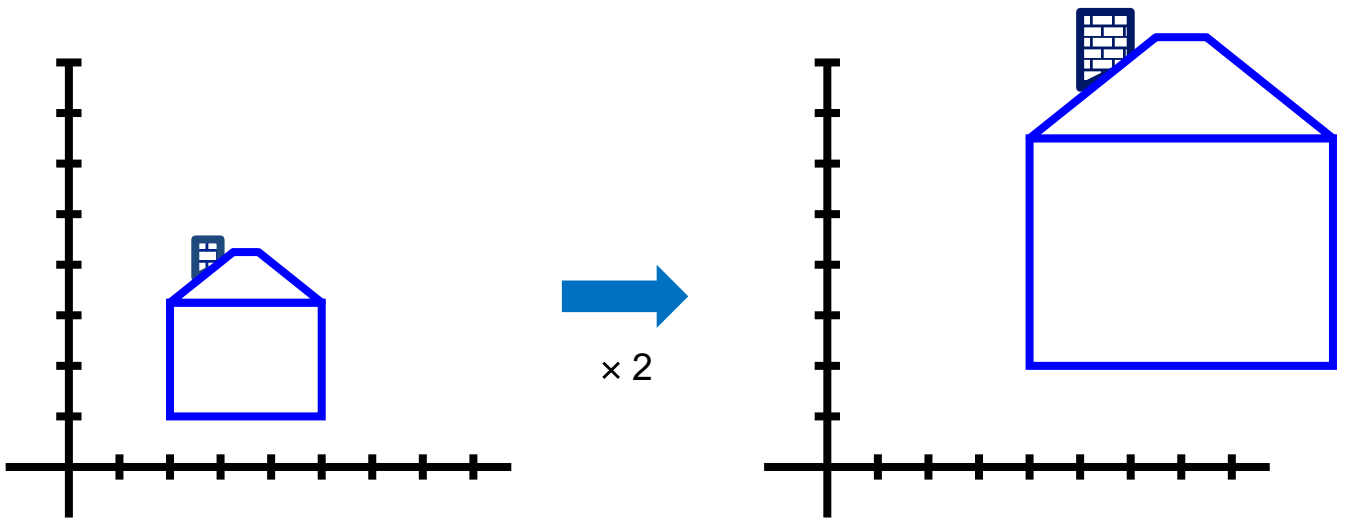


Perspective

[Slide credit (next few slides): A. Efros and/or S. Seitz]

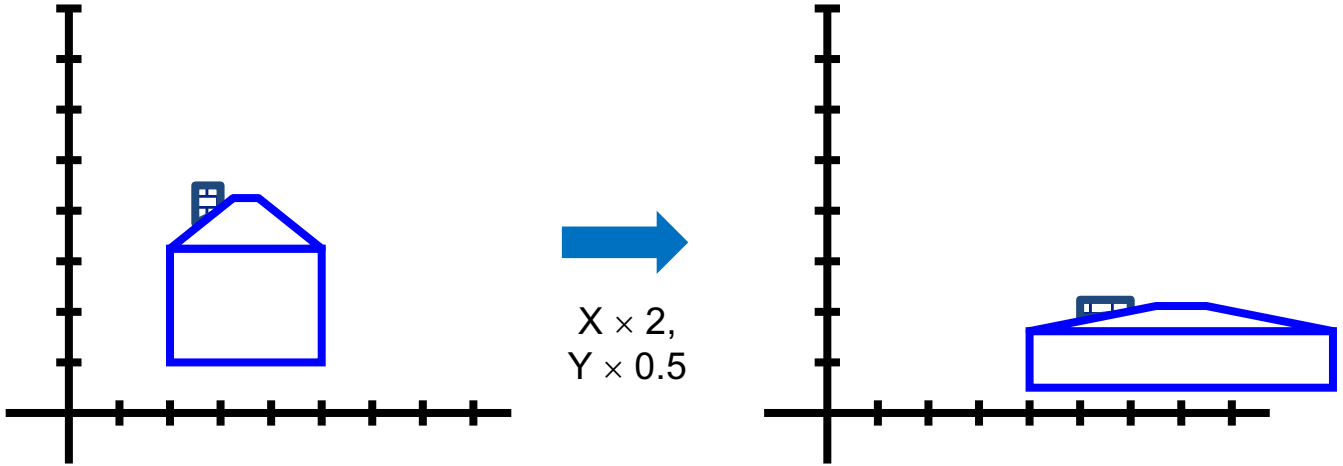
Scaling

- *Scaling* a coordinate means multiplying each of its components by a scalar
- *Uniform scaling* means this scalar is the same for all components:



Scaling

- *Non-uniform scaling*: different scalars per component:



Scaling

- Scaling operation: $x' = ax$

  $y' = by$

- Or, in matrix form:

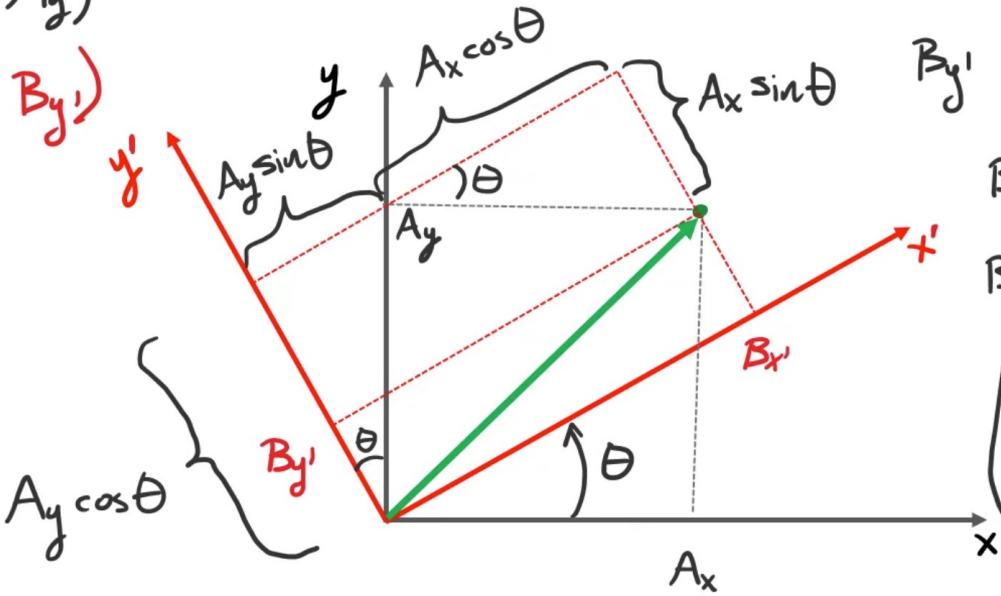
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}}_{\text{scaling matrix } S} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Rotation

Coordinate system transformation

$\vec{A} = (A_x, A_y)$

$\vec{B} = (B_{x'}, B_{y'})$



$$B_{x'} = A_y \sin\theta + A_x \cos\theta$$

$$B_{y'} = A_y \cos\theta - A_x \sin\theta$$

$$B_{x'} = \cos\theta A_x + \sin\theta A_y$$

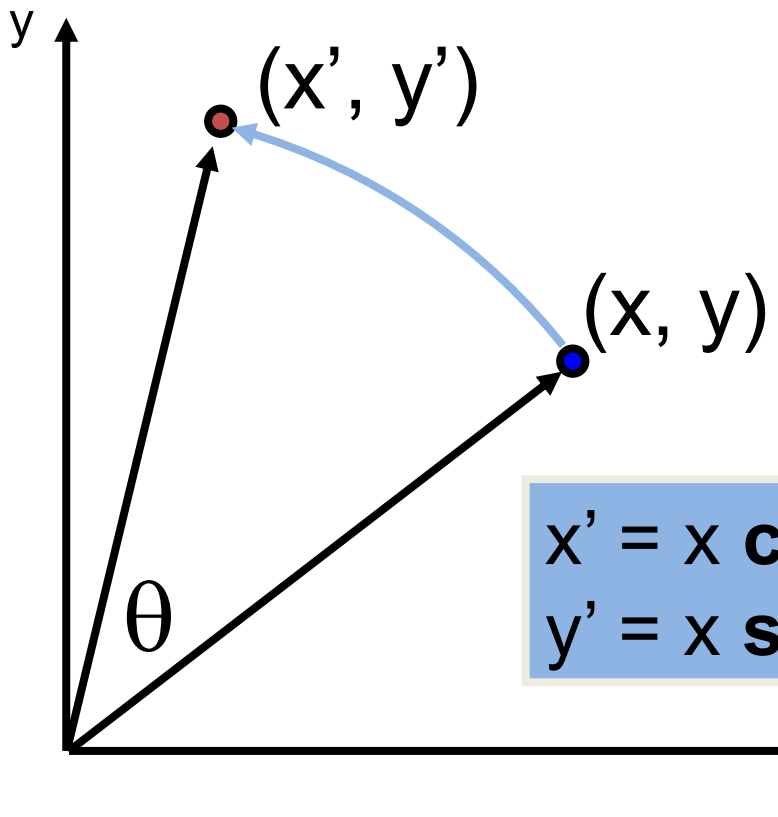
$$B_{y'} = -\sin\theta A_x + \cos\theta A_y$$

$$\begin{pmatrix} B_{x'} \\ B_{y'} \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} A_x \\ A_y \end{pmatrix}$$

Rotation matrix for axes

source -- https://www.youtube.com/watch?v=-HcDI_gyeMs

2D Rotation



Rotating a point with angle θ in the counter-clockwise direction is equivalent to rotating the axes with angle $-\theta$ (also in the counter-clockwise direction).

$$x' = x\cos(\theta) - y\sin(\theta)$$
$$y' = x\sin(\theta) + y\cos(\theta)$$

2D Rotation

We can represent rotation in matrix form (for rotating a point with angle θ in the counter-clockwise direction):

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{\mathbf{R}} \begin{bmatrix} x \\ y \end{bmatrix}$$

Even though sin(θ) and cos(θ) are nonlinear functions of θ,

- ***x' is a linear combination of x and y***
- ***y' is a linear combination of x and y***

What is the inverse of the transformation matrix for rotation?

- Rotating the point by $-\theta$ (which also means rotating the axes by θ)
- For rotation matrices $\mathbf{R}^{-1} = \mathbf{R}^{T}$ (because of orthogonality)

2D Translation

We use *Homogeneous coordinates to* represent translation as a 3x3 matrix.

$$x' = x + t_x$$

$$y' = y + t_y$$

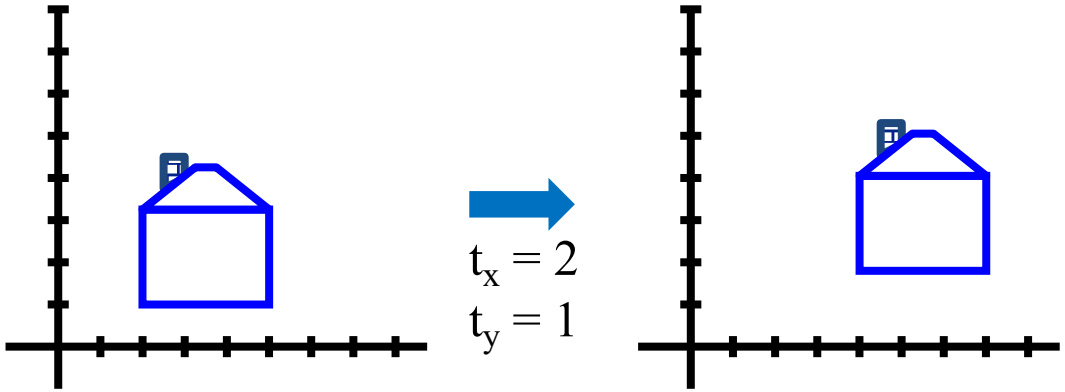
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation

- Example of translation

Homogeneous Coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$



Basic 2D Transformations



Original



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\Theta & -\sin\Theta & 0 \\ \sin\Theta & \cos\Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotate



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shear

Affine Transformations

- Affine transformations are combinations of …
  - Linear transformations, and
  - Translations
- Properties of affine transformations:
  - Origin does not necessarily map to origin (because of the translation term)
  - Lines map to lines
  - Parallel lines remain parallel
  - Ratios are preserved
  - Is closed under composition (i.e., combining multiple affine transformations also gives affine transformation)
  - Can model the change of basis/coordinate (e.g., from the world frame to the camera frame)

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} d_x \\ d_y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Affine: any transformation represented by a 3x3 matrix with last row [0 0 1]

Affine Transformations

Which ones are affine transformations?



Translation



Rotation



Scaling



Shear



Perspective

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} d_x \\ d_y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Affine: any transformation represented by a 3x3 matrix with last row [0 0 1]

Transformation Composition

- Transformations can be combined by matrix multiplication

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\Theta & -\sin\Theta & 0 \\ \sin\Theta & \cos\Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$\mathbf{p}'$ = $T(t_x,t_y)$ $R(\Theta)$ $S(s_x,s_y)$ $\mathbf{p}$

Translation Rotation Scale

note: you can also combine shear and reflection How to write reflection in matrix form?

Projective Transformations (a.k.a. Homographies)

- Projective transformations …
  - Affine transformations, and
  - Projective warps
- Properties of projective transformations:
  - Origin does not necessarily map to origin (because of the translation term)
  - Lines map to lines
  - Parallel lines do not necessarily remain parallel
  - Ratios are not preserved
  - Is closed under composition (i.e., combining multiple projective transformations also gives projective transformation)
  - Can model the change of basis/coordinate (e.g., from the world frame to the camera frame)

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

The last row is no longer [0 0 1]

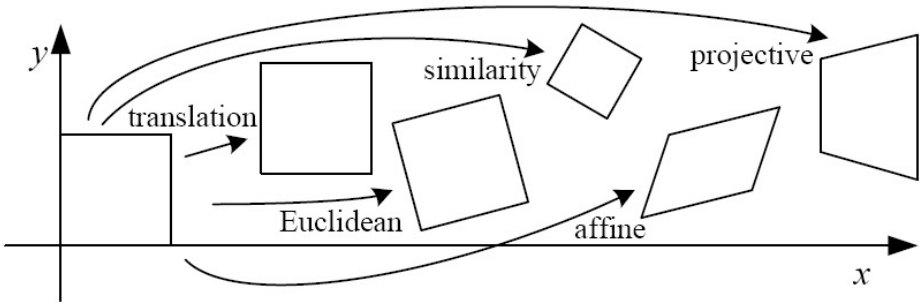


Original



Perspective

Basic 2D Transformations



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} \boldsymbol{I} \mid \boldsymbol{t} \end{bmatrix}_{2\times3}$	2	orientation + ⋯	
rigid (Euclidean)	$\begin{bmatrix} \boldsymbol{R} \mid \boldsymbol{t} \end{bmatrix}_{2\times3}$	3	lengths + ⋯	
similarity	$\begin{bmatrix} s\boldsymbol{R} \mid \boldsymbol{t} \end{bmatrix}_{2\times3}$	4	angles + ⋯	
affine	$\begin{bmatrix} \boldsymbol{A} \end{bmatrix}_{2\times3}$	6	parallelism + ⋯	
projective	$\begin{bmatrix} \tilde{\boldsymbol{H}} \end{bmatrix}_{3\times3}$	8	straight lines	

translation $\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$

rigid $\begin{bmatrix} \cos\theta & -\sin\theta & t_x \\ \sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$

similarity $\begin{bmatrix} s\cdot\cos\theta & -s\cdot\sin\theta & t_x \\ s\cdot\sin\theta & s\cdot\cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$

affine $\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$

projective $\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$

Projective Transformations (a.k.a. Homographies)

Why only 8 degrees of freedom for projective transformation?

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Hint: think about the following:

$$x'_{Euclidean} = \frac{ax + by + cw}{w'} = \frac{ax + by + cw}{gx + hy + iw} = \cdots$$

$$y'_{Euclidean} = \frac{dx + ey + fw}{w'} = \frac{dx + ey + fw}{gx + hy + iw} = \cdots$$

You can do something about this, and then the value stays the same

Agenda

- Global operations
  - Global Transformations
  - Image Warping
- Image Stitching
  - Transformation computation
    - Reprojection model
    - Key point matching
    - RANSAC
    - Solving transformation
  - Blending

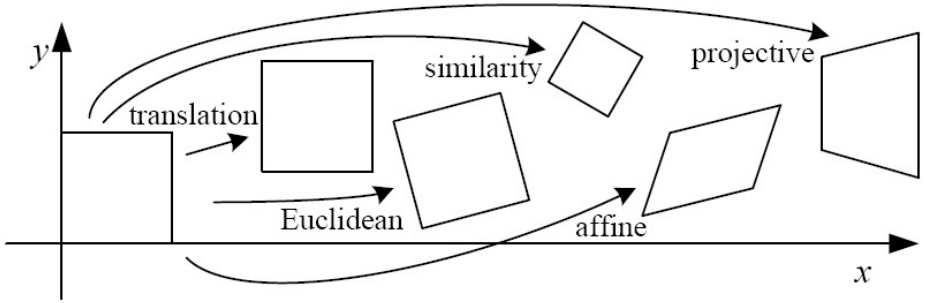
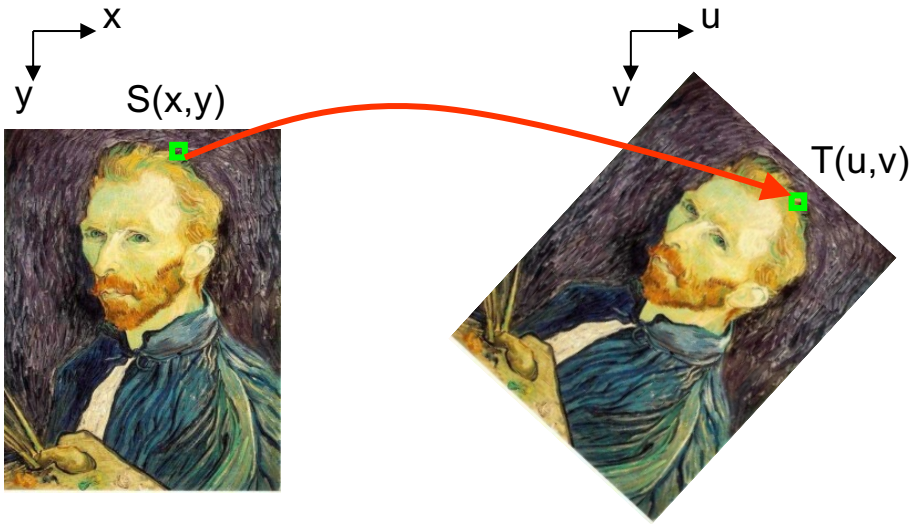




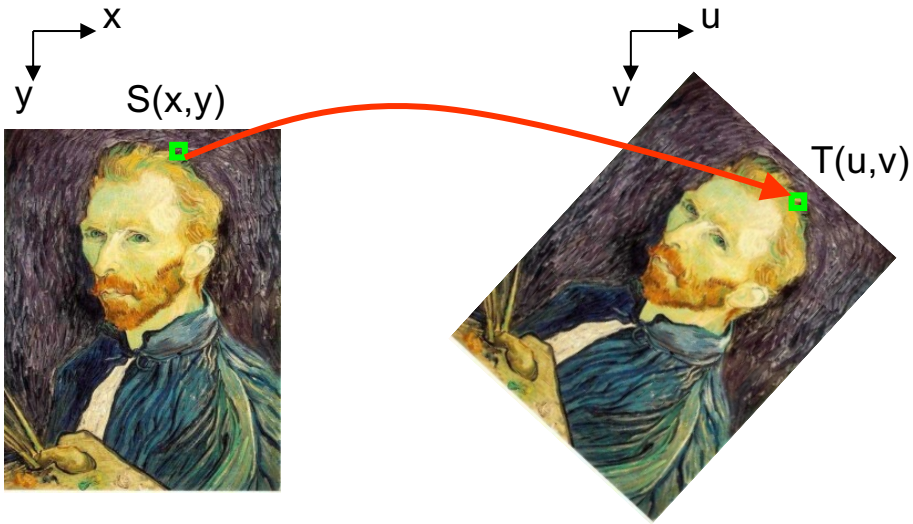
Image Warping



Given a coordinate transform function [f(x,y), g(x,y)] (or the inverse i.e., [F(u,v), G(u,v)]) and source image S(x,y), how do we compute a transformed target image T(u,v)?

Forward Warping

Imagine that we break the source image S into small pieces of puzzles, and then we want to put each piece to a new location to get the target image T. We apply the transformation function to find the new location.



Forward warping algorithm:

for $y = y_{min}$ to y_{max}

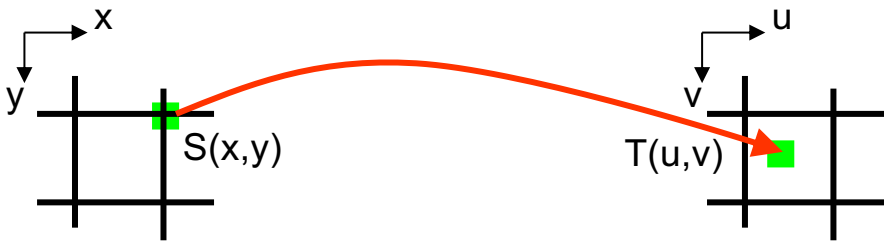
  for $x = x_{min}$ to x_{max}

    $u = f(x,y)$; $v = g(x,y)$

    copy pixel at source S(x,y) to T(u,v)

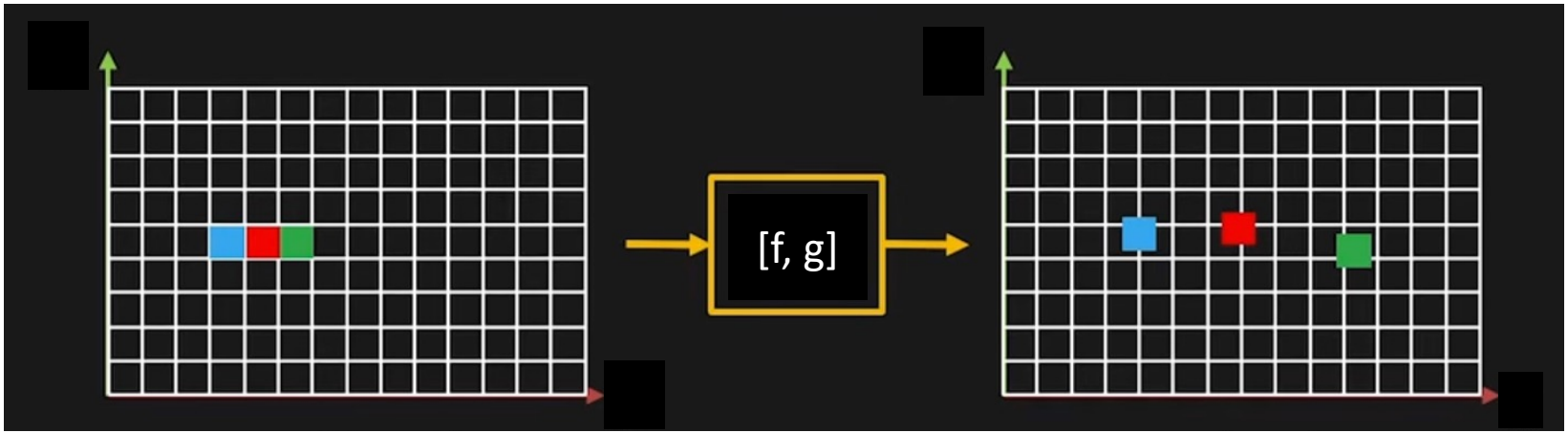
Any problems for forward warping?

Forward Warping - Problems



There could be pixels in T that are not filled with values

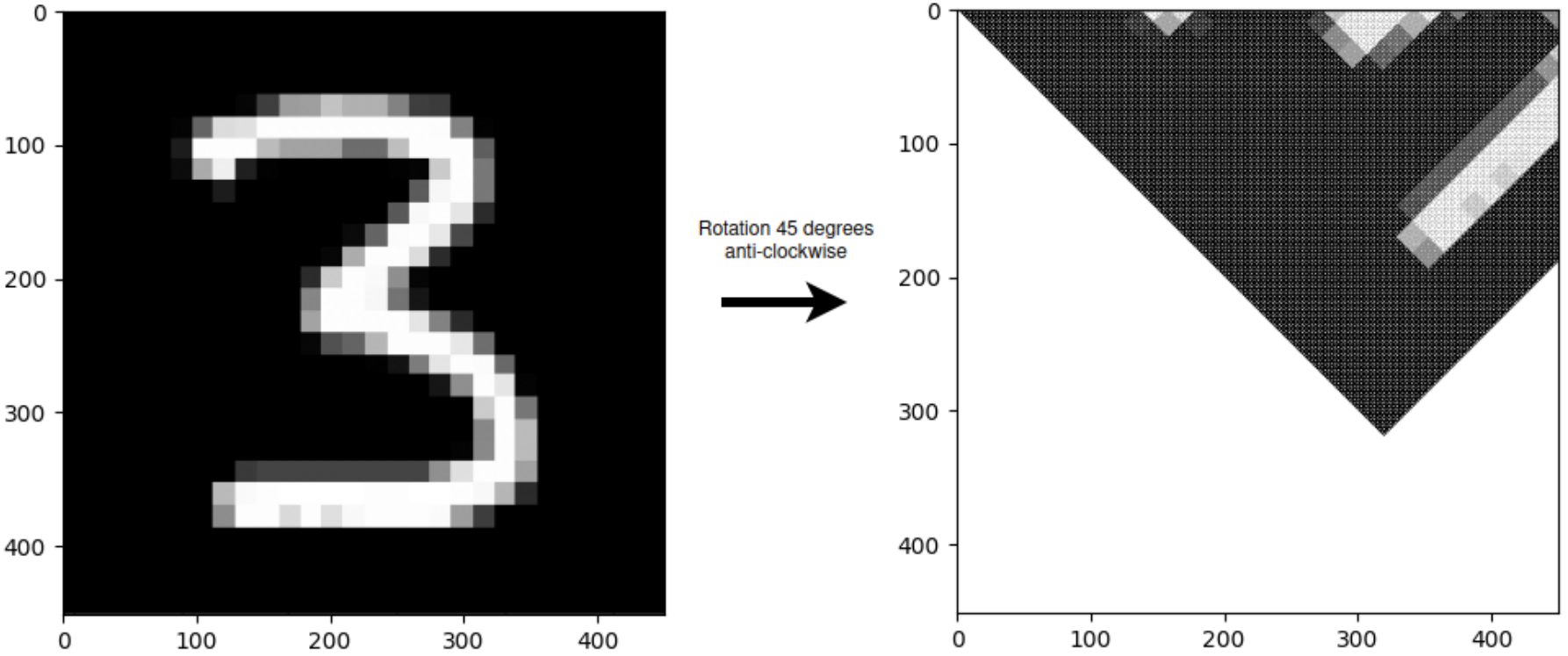
→ This gives holes in the warped image



source -- https://www.youtube.com/watch?v=D9rAOAL12SY&t=131s

Forward Warping - Problems

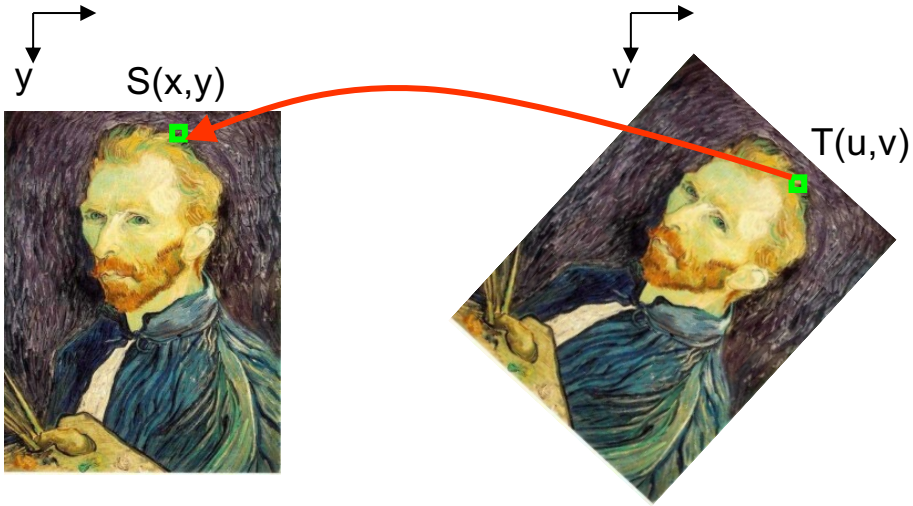
These holes are shown as white dots and strips in the image below



source -- https://towardsdatascience.com/forward-and-backward-mapping-for-computer-vision-833436e2472/

Inverse/Backward Warping

Imagine that we start from a blank sheet of paper (which is the target image T) with squares, and we want to fill out the color in these squares. We do a reverse transformation to find what are the pixel values that we should consider in the source image S.



Inverse warping algorithm:

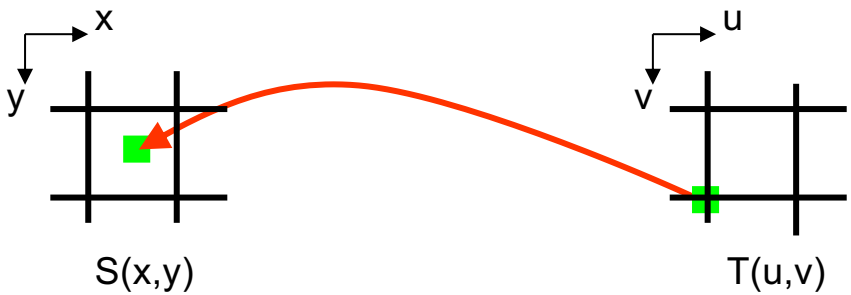
for $v = v_{min}$ to v_{max}

  for $u = u_{min}$ to u_{max}

    $x = F(u,v)$; $y = G(u,v)$

    copy pixel at source S(x,y) to T(u,v)

Image Warping



Q: What if pixel comes from “between” 4 pixels?

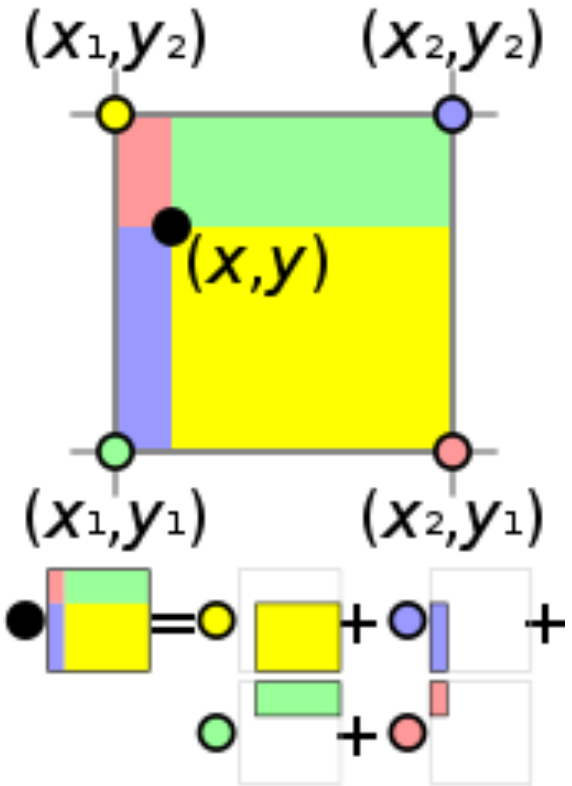
A: Interpolate color values from neighboring pixels, or just use the color value of the nearest neighboring pixel

Bilinear Interpolation

- **Goal:** Given an image with colors defined on integer grid locations, how to compute the color at a sub-pixel coordinate (x,y)?

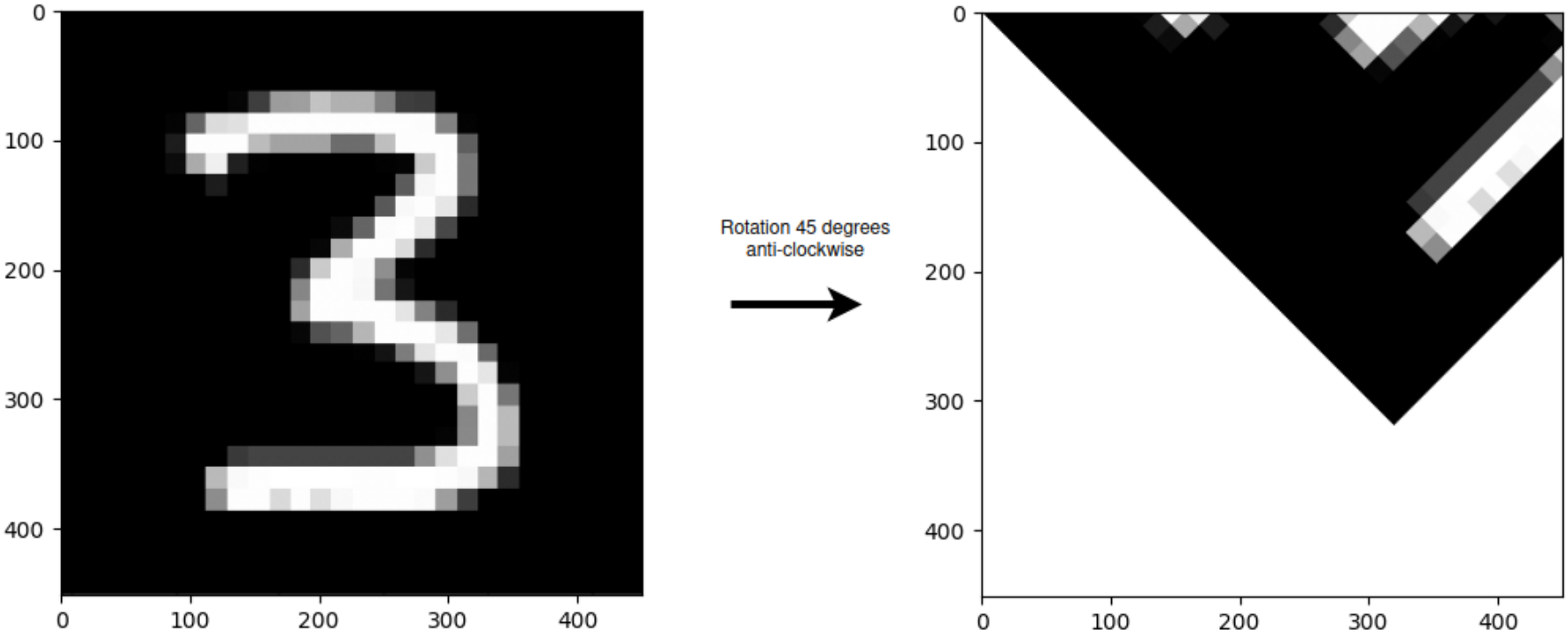
- **Approach:** The color of a point between discrete grid locations is defined as the weighted linear interpolation of the colors of the four neighboring pixels.

  The weights are determined based on the area of the rectangle between point (x,y) and the corresponding neighbor.



Backward Warping

Backward warping gives an image with no holes (pure black background below).



source -- https://towardsdatascience.com/forward-and-backward-mapping-for-computer-vision-833436e2472/

Agenda

- Global operations
  - Global Transformations
  - Image Warping
- Image Stitching
  - Transformation computation
    - Reprojection model
    - Key point matching
    - RANSAC
    - Solving transformation
  - Blending

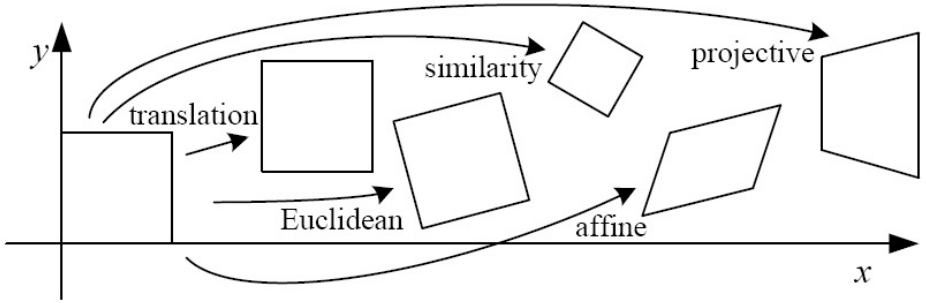




Image Stitching



Goal: Combine two or more overlapping images into one large image.

Typical Steps:

- Transformation computation
  - Reprojection model
  - Key point matching
  - RANSAC
  - Solving transformation
- Blending

[Image source: https://panoramic-photo-guide.com/best-photo-stitching-software-to-beginners.html]

Image Reprojection Model

Image 1

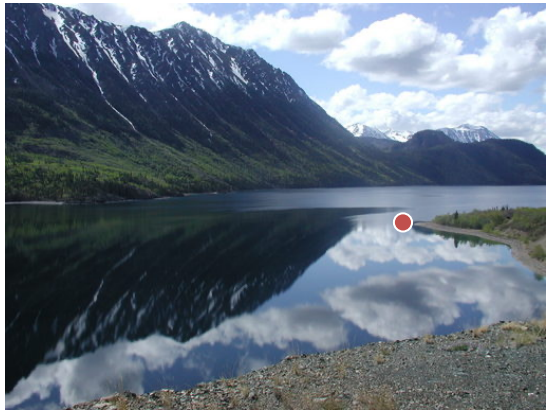
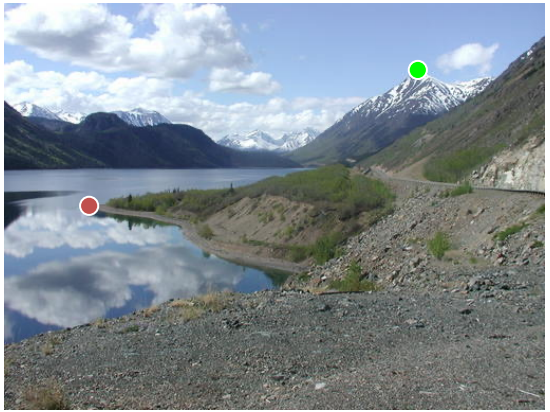
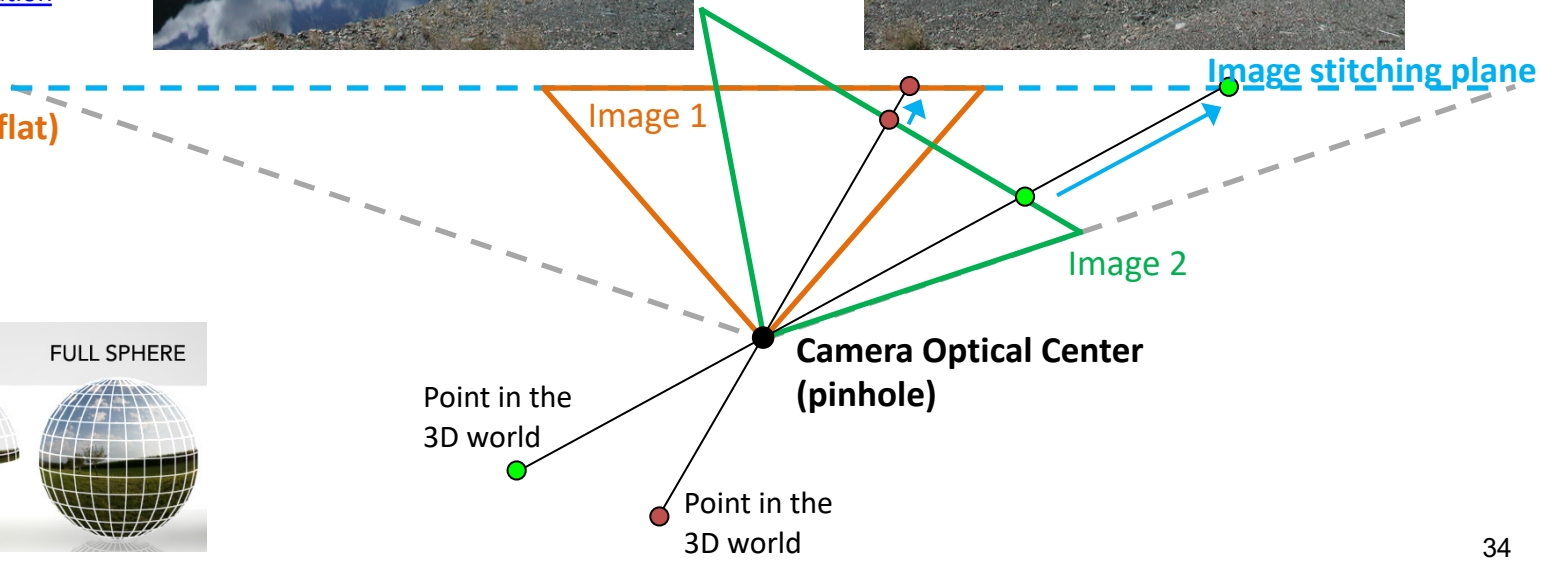


Image 2

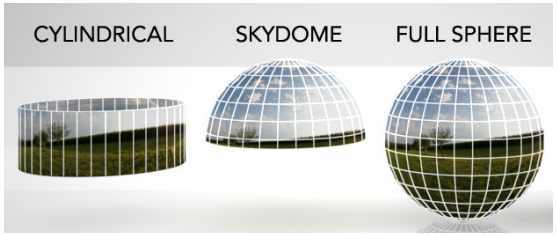


We assume that the optical center is the same for both images (only camera rotation and no translation). This guarantees homography between image 1 and 2. See here for a proof -- https://visionbook.mit.edu/homography.html#camera-rotation

Planar projection:
(i.e., image plane is flat)



There are also other projection models:



Homography

Another way to guarantees homography is when two cameras observe a 2D plane.

See here for a proof -- https://visionbook.mit.edu/homography.html#camera-rotation (also the source of the following images)



Image 1



Image 2

Keypoint detection

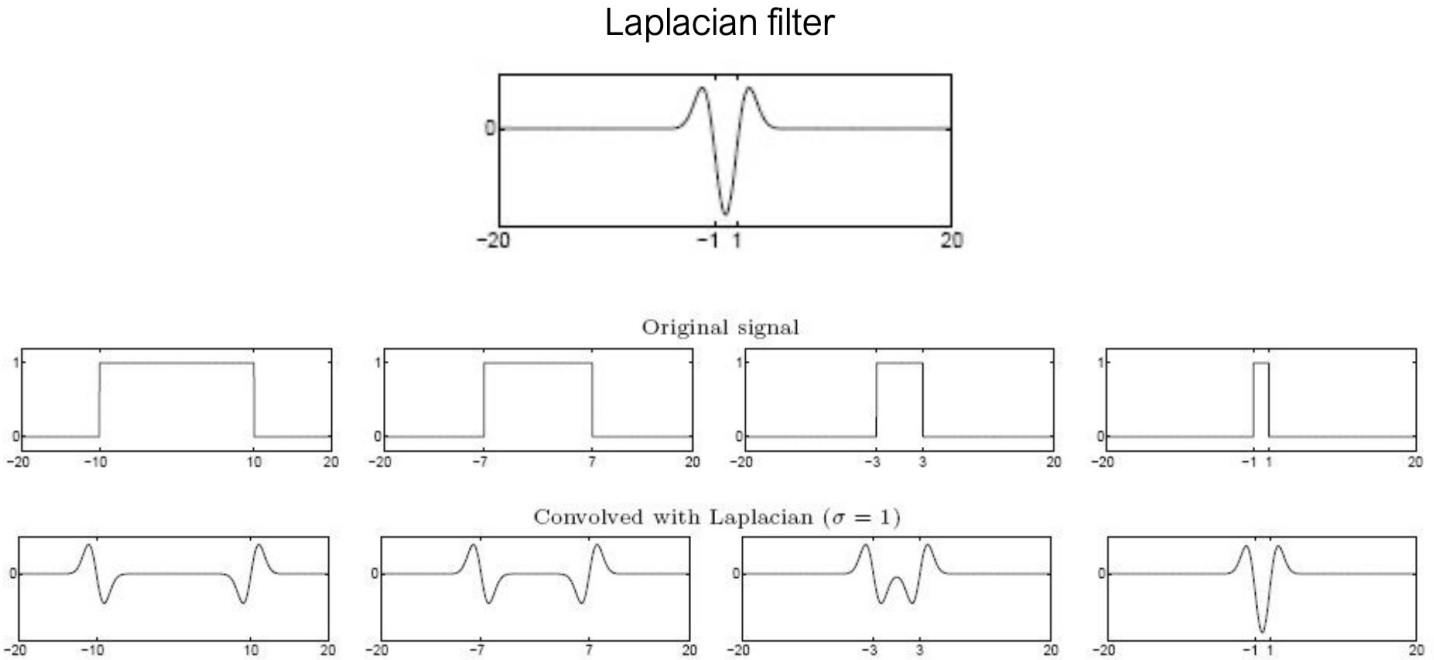
Can we detect the same interest points despite changes in image scale?





Find blobs as keypoint candidates

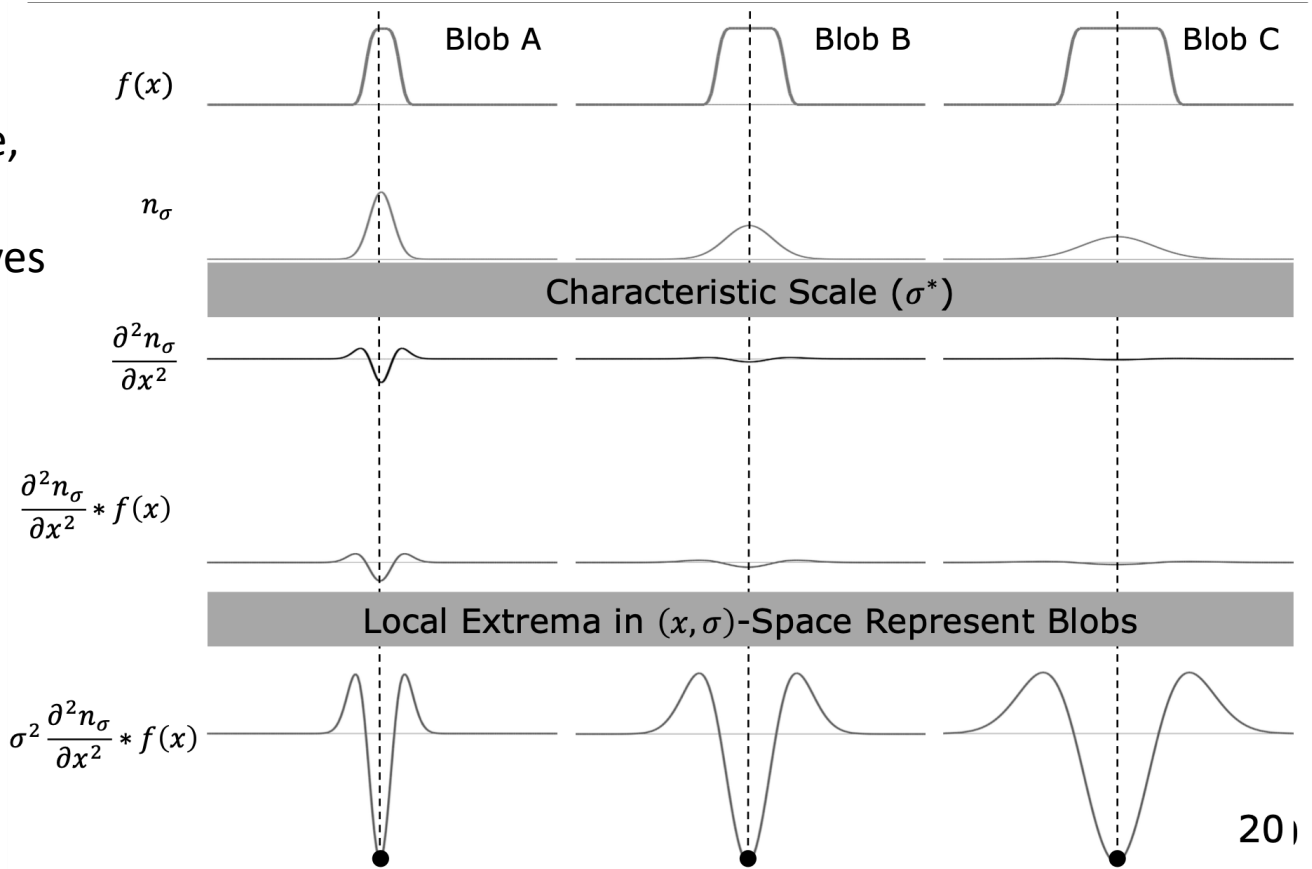
A LoG (Laplacian of Gaussian) filter with some variance σ can give a peak response after convolution for some “interesting” signal at a certain scale.



Source -- https://www.cs.cmu.edu/~16385/lectures/lecture5.pdf

Find blobs as keypoint candidates

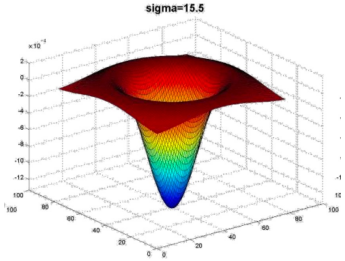
For some “interesting” signal at a different scale, there is a LoG filter with some variance σ that gives a peak response after convolution.



Source -- https://www.youtube.com/watch?v=zItstOggP7M

Find blobs as keypoint candidates

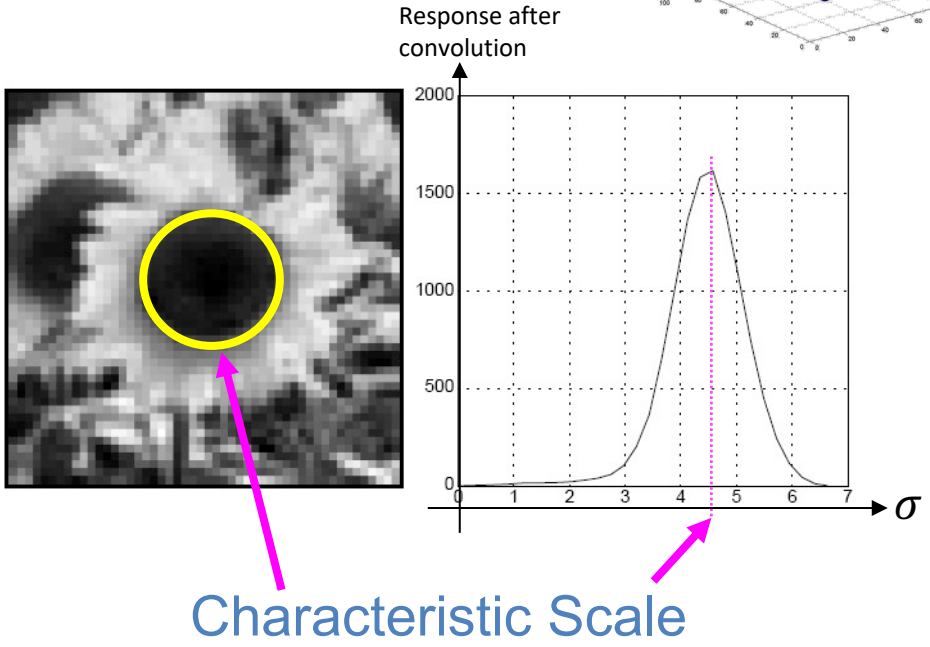
Characteristic Scale → The scale that produces peak of response



Procedure:

- Convolve image with Laplacian of Gaussians (LoG) of varying scale
- Detect max responses across scale space

A narrower or wider LoG would have a weaker response for this flower.



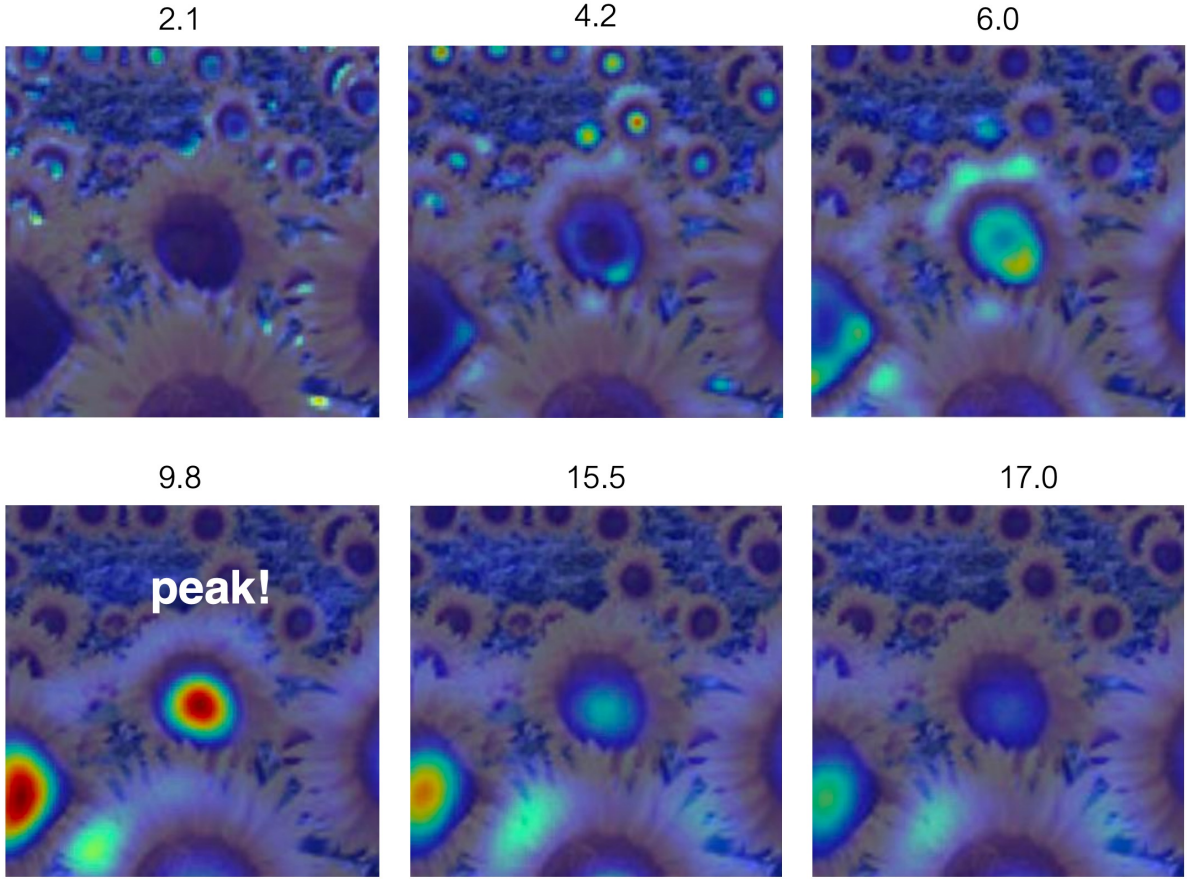
Slide credit:
Lana Lazebnik

Find blobs as keypoint candidates



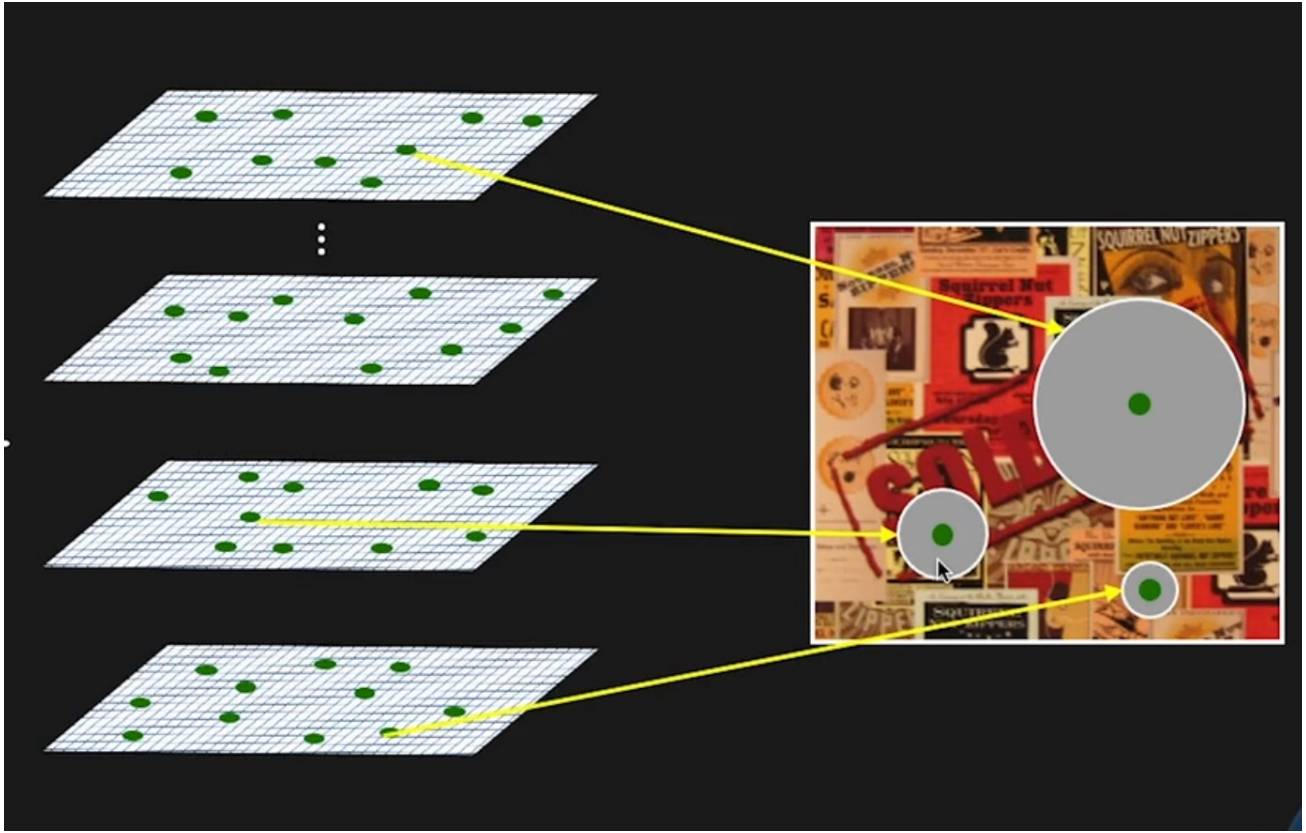
We can find the blob by convolving LoG (with different variances) to the same image and then identify the peak response.

Source -- https://www.cs.cmu.edu/~16385/lectures/lecture5.pdf



SIFT (Scale-invariant feature transform) blobs

Find interesting points and blobs at different scale using different LoGs (more precisely, using Difference of Gaussians as an approximation)

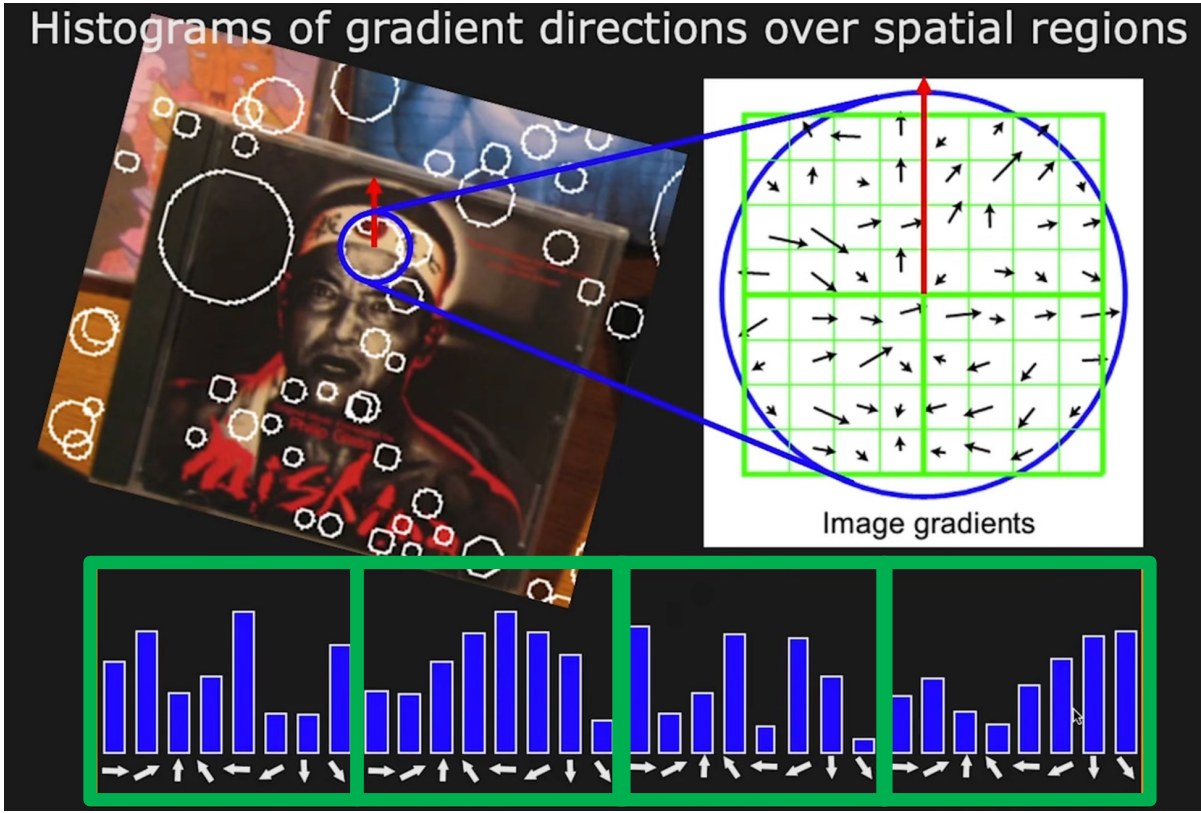


Source -- https://www.youtube.com/watch?v=ram-jbLJjFg

SIFT descriptor



Compute histogram of gradients for a local patch, then rotate the image using the dominate gradient direction, and finally put them together into a vector.

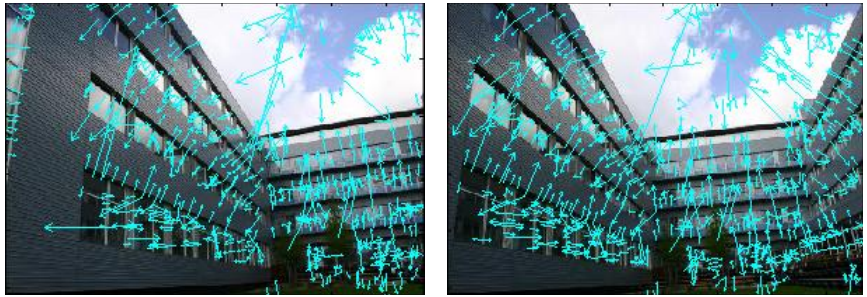


Source -- https://www.youtube.com/watch?v=IBcsS8_gPzE

Keypoint Matching



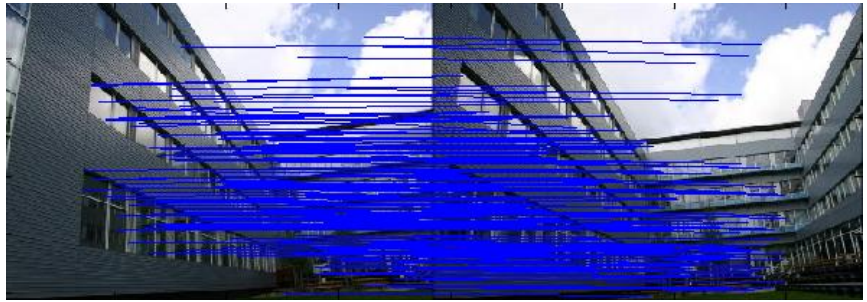
Input Images



SIFT Features

For the SIFT feature in image A (a vector in a high dimensional space), we can:

- find its nearest neighbor of the SIFT feature in image B (in the same high dimensional space) using a distance metric (e.g., L2 distance)
- or use the Lowe's ratio test (comparing the distances to the 1st vs. 2nd nearest neighbor, and check if it is smaller than a threshold)



Nearest Neighbor Matches

[Image source: Cansin Yildiz]

Keypoint Matching

Why using Lowe's ratio test to compare the distances to the 1st vs. 2nd nearest neighbor?

Hint: think about the following situation (Is it a good key point? Why?)

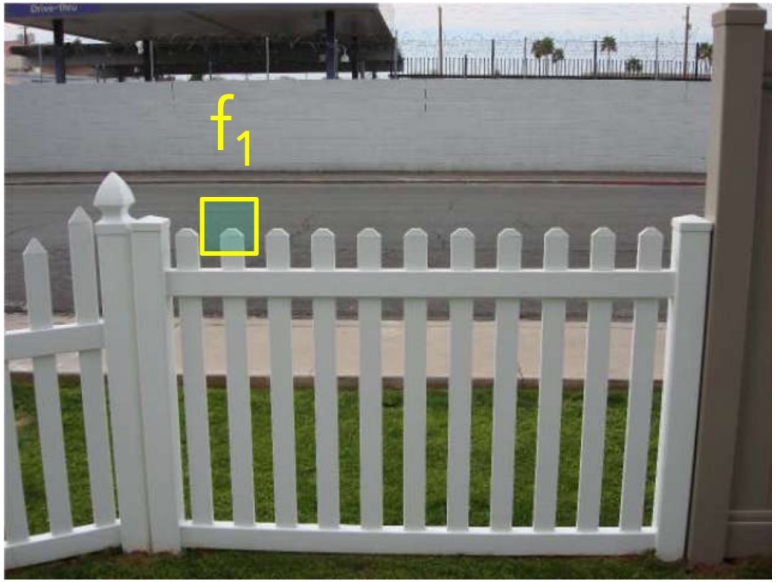


image A

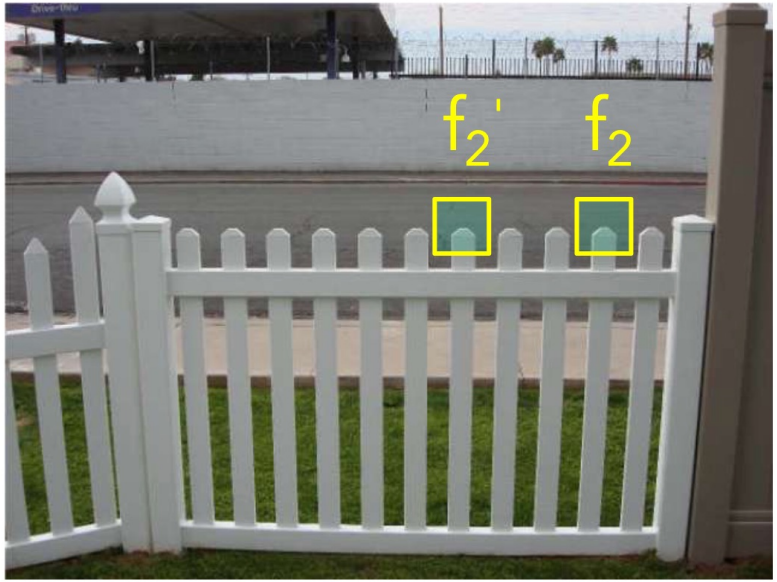
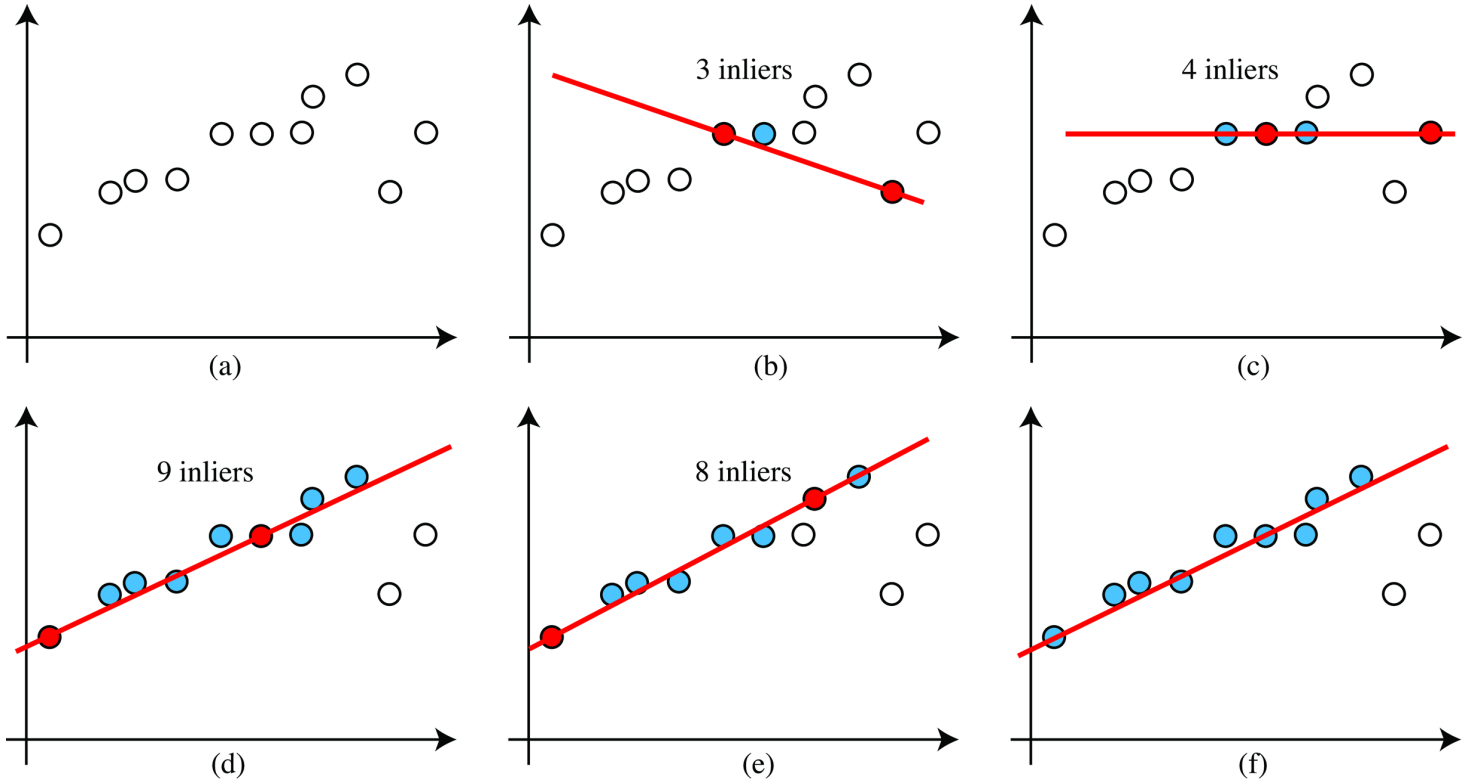


image B

source -- https://www.cs.cornell.edu/courses/cs5670/2025fa/slides/lec20-fitting.pdf

Concept of RANSAC (Reminder)

Two points are sufficient to estimate a line. We want the line with the max number of inliers

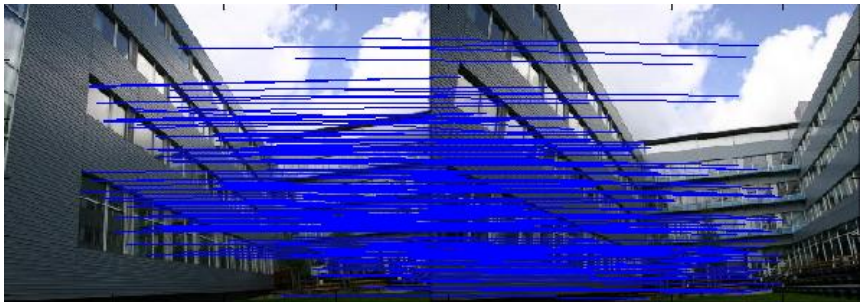


source -- https://visionbook.mit.edu/homography.html#sec-RANSAC

RANSAC for homography

- Describe a Homography:

  Points from images a and b are related by $p_b = H_{ab}\, p_a$

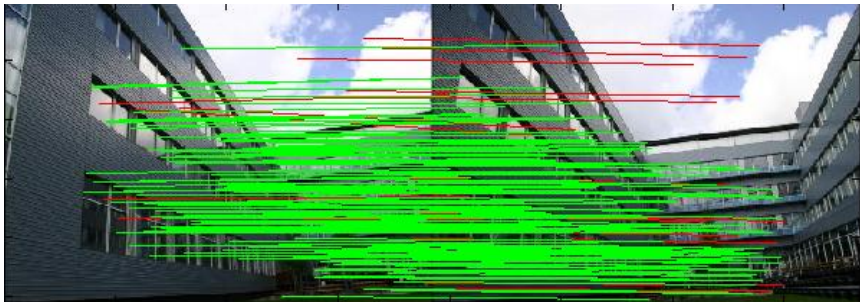


Nearest Neighbor Matches

Iterate N times: Why four?

1. Randomly select four feature pairs
2. Compute exact homography H
3. Compute inliers: $\mathbf{dist}(\boldsymbol{p_b}, \boldsymbol{H_{ab}\, p_a}) < \epsilon$ (e.g., L2 distance)
4. Keep largest set of of inliers

Finally, compute least-squares homography estimate on all inliers



Inliers and Outliers

[Image source: Cansin Yildiz]

Why 4 keypoint pairs is sufficient?

Write out the linear equation for homography and rearrange the terms:

$$\begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} \cong \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

Homography H

How many degrees of freedom in matrix H? (i.e., How many unknown variables?)

$$x_i' = \frac{h_{00}x_i + h_{01}y_i + h_{02}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$y_i' = \frac{h_{10}x_i + h_{11}y_i + h_{12}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

We have two equations per keypoint pair

$$x_i'(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$

$$y_i'(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

RANSAC for homography

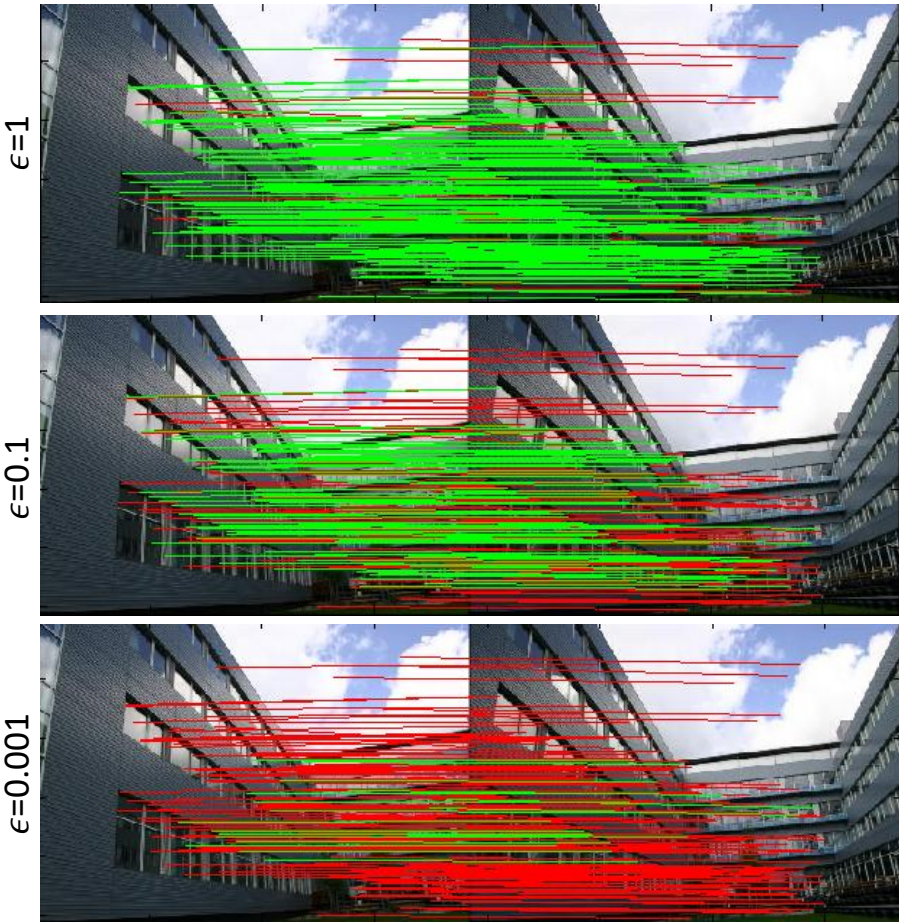
- Various inlier thresholds ϵ:

  Points from images a and b are related by $p_b = H_{ab}\, p_a$

Iterate N times:

1. Randomly select four feature pairs
2. Compute exact homography H
3. Compute inliers: $\mathbf{dist}(\boldsymbol{p_b}, \boldsymbol{H_{ab}\, p_a}) < \boldsymbol{\epsilon}$
4. Keep largest set of of inliers

Finally, compute least-squares homography estimate on all inliers



[Image source: Cansin Yildiz]

Final Stitching Result



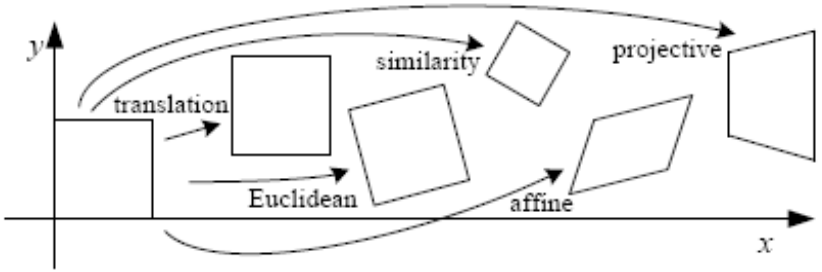
Input Images



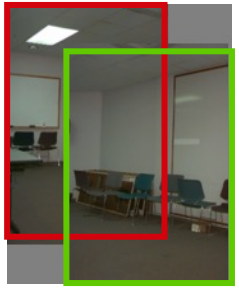
Stitched Images

[Image source: Cansin Yildiz]

Image Transformation Models

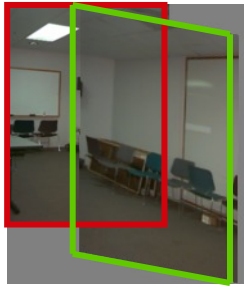


Translation



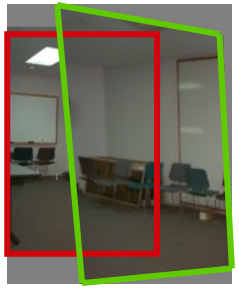
2 unknowns

Affine



6 unknowns

Projective



8 unknowns

You can use different transformation models. Here, we are talking about the projective transformation model.

Solving for Homographies

Write out the linear equation for homography and rearrange the terms:

$$\begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} \cong \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

$$x_i' = \frac{h_{00}x_i + h_{01}y_i + h_{02}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$y_i' = \frac{h_{10}x_i + h_{11}y_i + h_{12}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$x_i'(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$
$$y_i'(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

$$x_i'(h_{20}x_i + h_{21}y_i + h_{22}) - h_{00}x_i - h_{01}y_i - h_{02} = 0$$
$$y_i'(h_{20}x_i + h_{21}y_i + h_{22}) - h_{10}x_i - h_{11}y_i - h_{12} = 0$$

Solving for Homographies

$$x_i'(h_{20}x_i + h_{21}y_i + h_{22}) - h_{00}x_i - h_{01}y_i - h_{02} = 0$$
$$y_i'(h_{20}x_i + h_{21}y_i + h_{22}) - h_{10}x_i - h_{11}y_i - h_{12} = 0$$

This measures how much one keypoint pair disagrees with the homography

Rewrite the equations in matrix form:

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x_i'x_i & -x_i'y_i & -x_i' \\ 0 & 0 & 0 & x_i & y_i & 1 & -y_i'x_i & -y_i'y_i & -y_i' \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Linear System for Homographies

Stack together the equations from multiple keypoint matching pairs into a linear system $\mathrm{Ah} = 0$

This measures how much all keypoint pairs disagree with the homography

$$\underbrace{\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1 x_1 & -x'_1 y_1 & -x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1 x_1 & -y'_1 y_1 & -y'_1 \\ & & & & & \vdots & & & \\ x_n & y_n & 1 & 0 & 0 & 0 & -x'_n x_n & -x'_n y_n & -x'_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -y'_n x_n & -y'_n y_n & -y'_n \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix}}_{\mathbf{h}} = \underbrace{\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}}_{\mathbf{0}}$$

$\mathbf{A}$: 2n × 9 — $\mathbf{h}$: 9 x 1 — $\mathbf{0}$: 2n x 1

Linear System for Homographies

To solve $\mathrm{Ah} = 0$, we cast it as a least squares problem:

$$\min_{\mathrm{h}} \|\mathrm{Ah} - 0\|^2$$

Since h is only defined up to a scale, we add a constraint that h needs to be a unit vector. This can also avoid a trivial solution $\mathrm{h} = 0$, so we have a constrained optimization problem:

$$\min_{\mathrm{h}} \|\mathrm{Ah}\|^2 \quad \text{such that} \quad \|\mathrm{h}\|^2 = 1$$

Expand the terms, we get:

$$\min_{\mathrm{h}} (\mathrm{h}^T \mathrm{A}^T \mathrm{Ah}) \quad \text{such that} \quad \mathrm{h}^T \mathrm{h} = 1$$

We use a Lagrange multiplier λ to turn this into an unconstrained version:

$$\min_{\mathrm{h},\lambda} (\mathrm{h}^T \mathrm{A}^T \mathrm{Ah}) - \lambda(\mathrm{h}^T \mathrm{h} - 1)$$

Linear System for Homographies

Now we have an unconstrained optimization problem:

$$\min_{\mathrm{h},\lambda} L(\mathrm{h},\lambda) = \min_{\mathrm{h},\lambda}(\mathrm{h}^T\mathrm{A}^T\mathrm{A}\mathrm{h}) - \lambda(\mathrm{h}^T\mathrm{h} - 1)$$

Take the derivatives and set it to zero. Then, we get a classical eigenvalue problem:

$$\frac{\partial L}{\partial \mathrm{h}} = 2\mathrm{A}^T\mathrm{A}\mathrm{h} - 2\lambda\mathrm{h} = 0 \quad \Rightarrow \quad \mathrm{A}^T\mathrm{A}\mathrm{h} = \lambda\mathrm{h}$$

The solutions are pairs of eigenvalue λ and eigenvector h of matrix $\mathrm{A}^T\mathrm{A}$

Recall that we want to minimize the objective function $\mathrm{h}^T\mathrm{A}^T\mathrm{A}\mathrm{h}$.

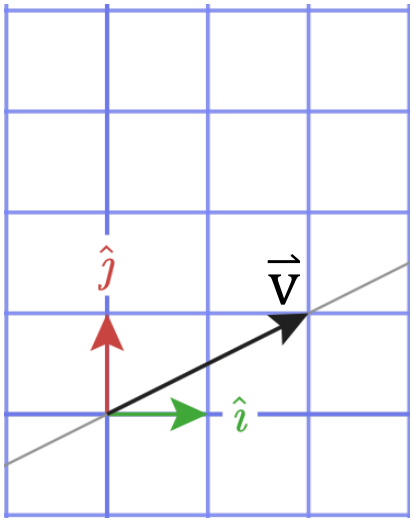
Since the critical points (min, max, saddle points) satisfy $\mathrm{A}^T\mathrm{A}\mathrm{h} = \lambda\mathrm{h}$, we know that objective function at a critical point should have value $\mathrm{h}^T\mathrm{A}^T\mathrm{A}\mathrm{h} = \mathrm{h}^T\lambda\mathrm{h} = \lambda$, since h is a unit vector.

So, we want the eigenvector h of matrix $\mathrm{A}^T\mathrm{A}$ with the smallest eigenvalue λ

see here for the eigenvalue problem -- https://www.3blue1brown.com/lessons/eigenvalues/

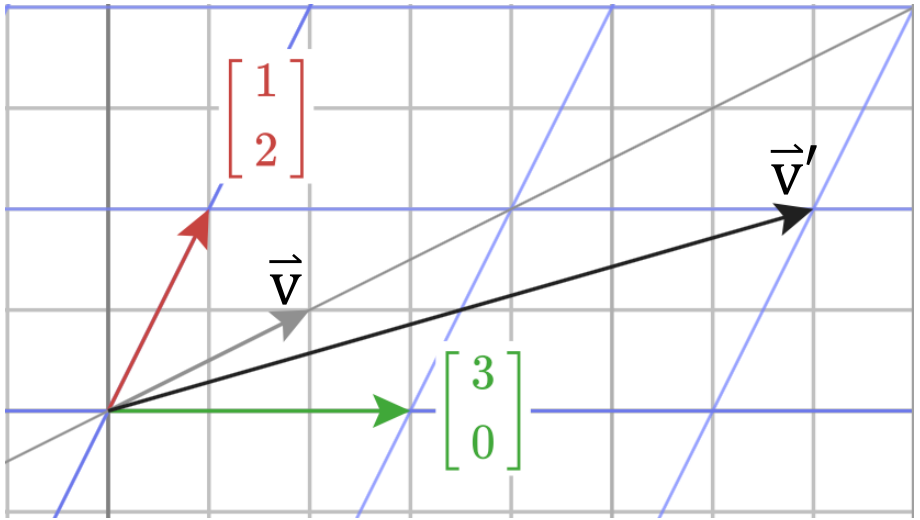
Quick reminder of eigenvalue problem

Vectors will typically be changed after transformation using matrix M.



$$M\vec{v} = \vec{v}'$$

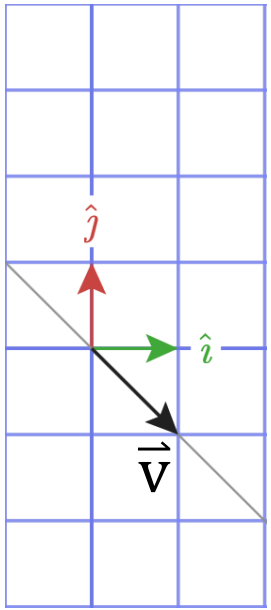
$$M = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$$



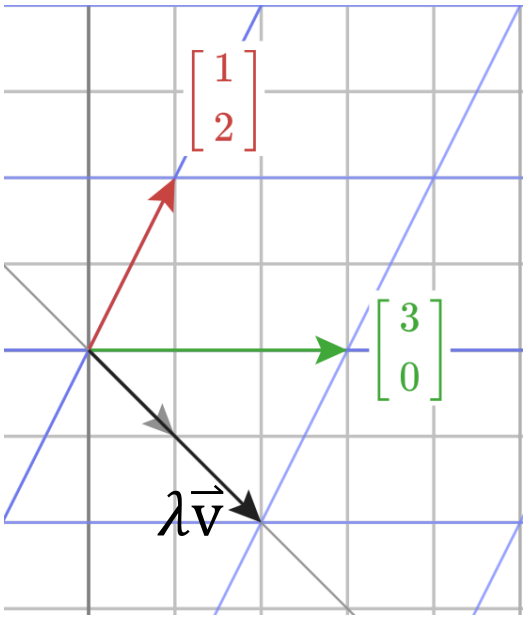
source -- https://www.3blue1brown.com/lessons/eigenvalues/

Quick reminder of eigenvalue problem

Eigenvectors of matrix M are a set of unit vectors $\vec{\mathrm{v}}$ where their directions remain unchanged after being transformed by M and is just scaled by eigenvalue λ.



$$\mathrm{M}\vec{\mathrm{v}} = \lambda\vec{\mathrm{v}}$$



see here about how to compute eigenvectors and eigenvalues -- https://www.3blue1brown.com/lessons/eigenvalues/

Quick reminder of eigenvalue problem

$$M\vec{v} = \lambda\vec{v} \Rightarrow (M - \lambda I)\vec{v} = 0$$

$$\Rightarrow \det(M - \lambda I) = 0$$

$\Rightarrow$ Find the roots of the polynomial function, which are λ

$\Rightarrow$ For each λ, solve $(M - \lambda I)\vec{v} = 0$

Example

$$M = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \Rightarrow \det\begin{bmatrix} 3-\lambda & 1 \\ 0 & 2-\lambda \end{bmatrix} = (3-\lambda)(2-\lambda) = 0 \Rightarrow \lambda = 2 \text{ or } 3$$

$$\lambda = 2 \Rightarrow \begin{bmatrix} 3-2 & 1 \\ 0 & 2-2 \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \text{ and } x^2 + y^2 = 1 \Rightarrow \vec{v} = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

see here about how to compute eigenvectors and eigenvalues -- https://www.3blue1brown.com/lessons/eigenvalues/

Agenda

- Global operations
  - Global Transformations
  - Image Warping
- Image Stitching
  - Transformation computation
    - Reprojection model
    - Key point matching
    - RANSAC
    - Solving transformation
  - Blending

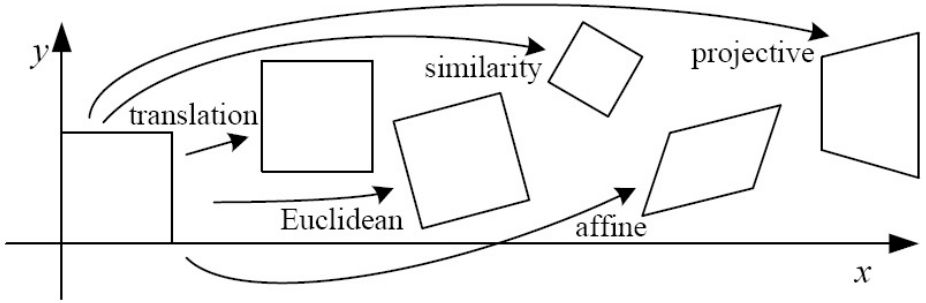
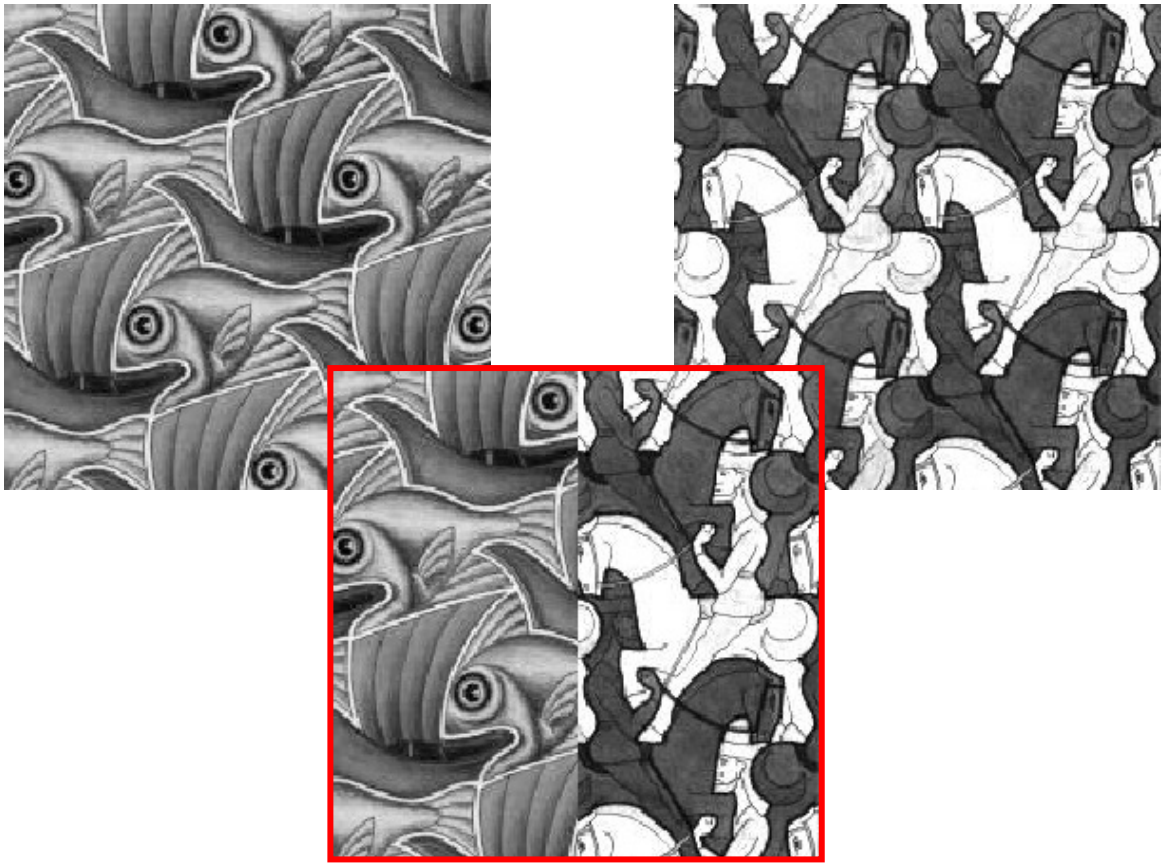
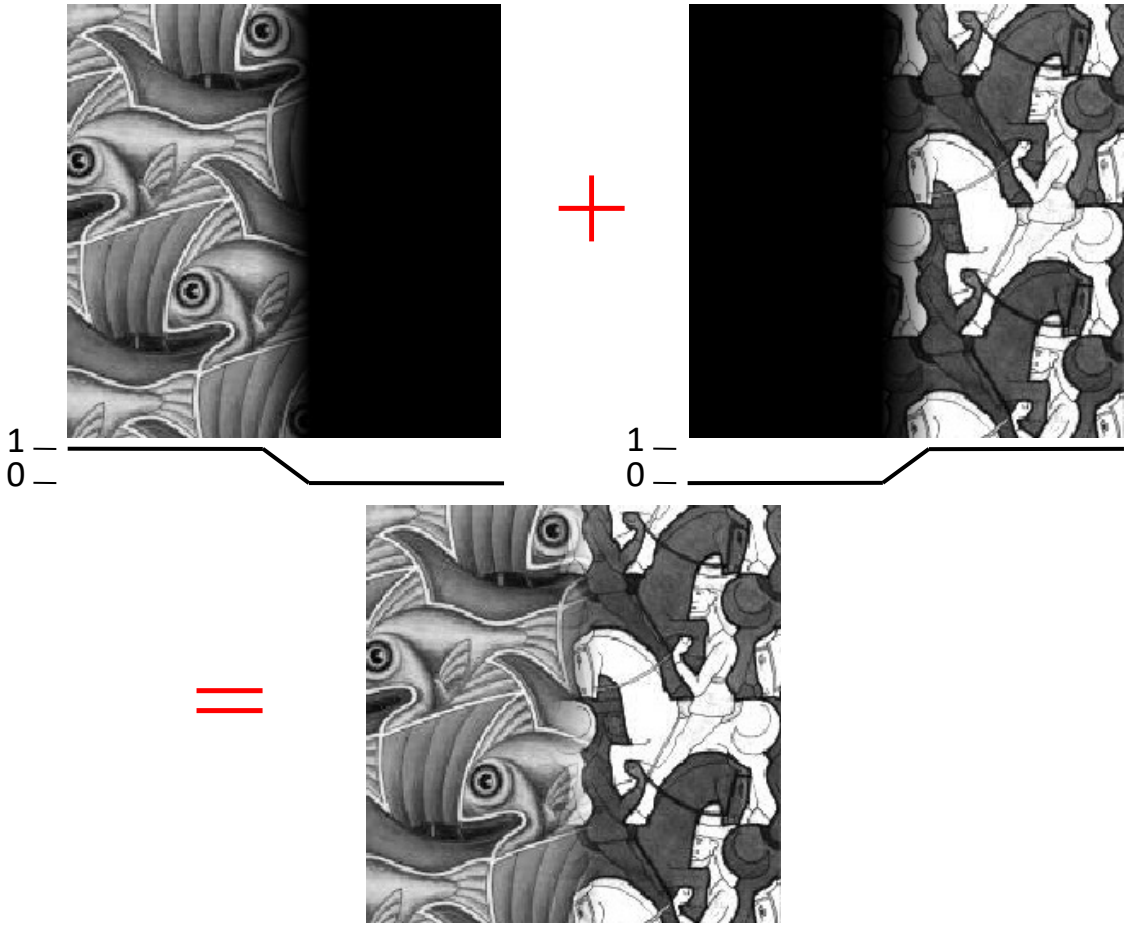




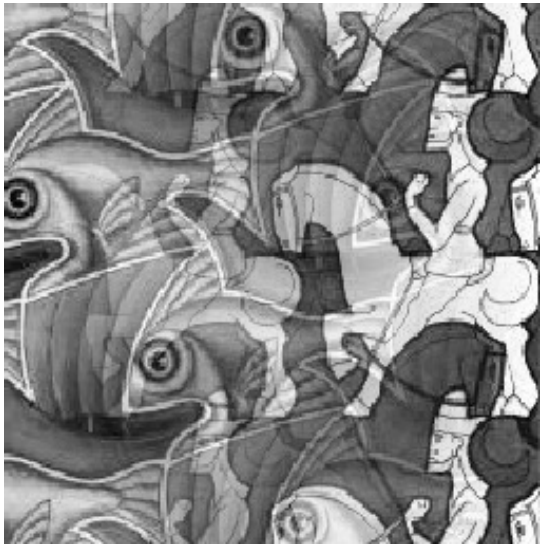
Image Blending

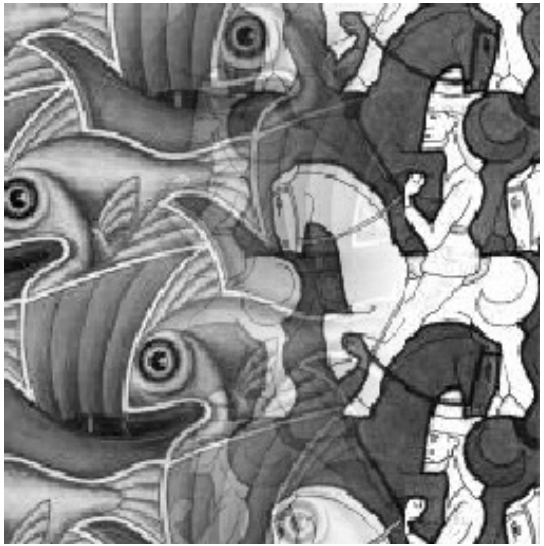


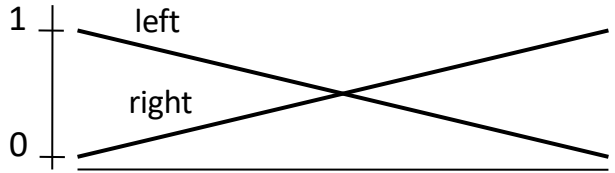
Weight blending

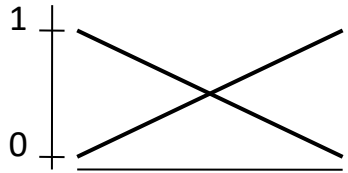


Effect of Window (ramp-width) Size



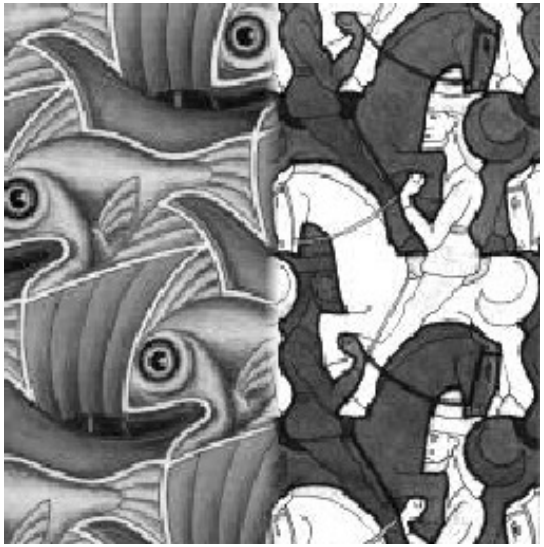


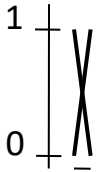


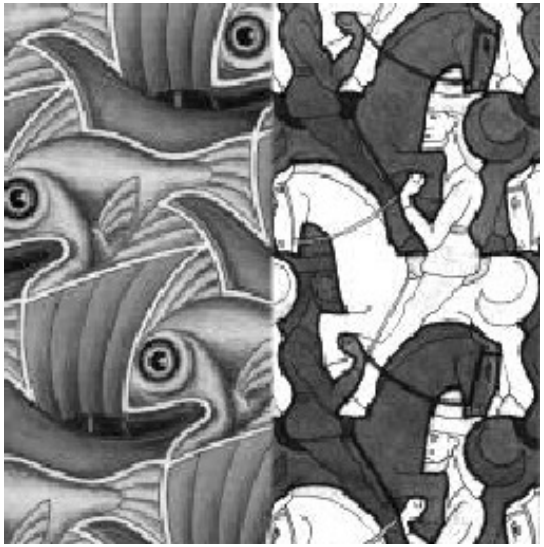


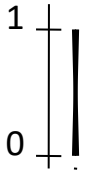
“cross-fading”

Effect of Window (ramp-width) Size

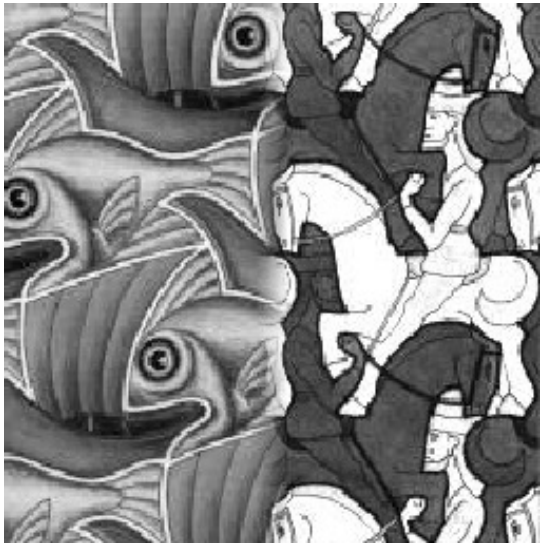








Good Window Size



There are also other ways of blending images. Here we only mention one method.

Resources

- Affine transformations in 5 minutes
- Warping and Blending Images | Image Stitching
- The determinant | Chapter 6, Essence of linear algebra
- Eigenvectors and eigenvalues | Chapter 14, Essence of linear algebra

Questions?

Disclaimer

Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.